



Degree project in Master's Programme, Systems, Control and Robotics

Second cycle, 30 credits

Real-time motion planning of 6 DOF Collaborative Robot

SEYEDHESAM AHMADI

Real-time motion planning of 6 DOF Collaborative Robot

SEYEDHESAM AHMADI

Master's Programme, Systems, Control and Robotics, 120 credits
Date: January 24, 2022

Supervisors: Jörg Conradt, Sakari Salmi
Examiner: Arvind Kumar
School of Electrical Engineering and Computer Science
Host company: Ericsson AB

Abstract

Motion planning is an essential component of an autonomous system. This project aims to design a motion planning module to automate the screwing process of radio units. The goal is to choose and implement a motion planner that provides the speed, precision, and efficiency required for the screwing task on a radio filter with a large number of holes located close to each other.

Four control-based motion planners were investigated on a 6 Degrees Of Freedom (DOF) robot arm in Robot Operating System (ROS). The investigated motion planners are Rapidly-exploring Random Trees, The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration (KPIECE), The Path-Directed subdivision Trees (PDST), Expansive Space Trees (EST). All these planners are available in The Open Motion Planning Library (OMPL). The motion planners were implemented on a simulated version of a UR5 robot arm. This simulated model is generated by the MoveIt Setup Assistant, which is the primary tool for creating configuration files for kinematics chains in MoveIt.

ROS is the chosen platform to design various control methods and motion planning algorithms. Hence two primary workspaces have been created. These workspaces contain several packages and nodes with multiple tasks such as motion planning, visualization, and data extraction. All the nodes communicate using ROS communication tools such as messages services and action client services.

Furthermore, this project covers also test and benchmarking all the mentioned planners to determine which planner provides optimal performance in different environments. The planner's performance is evaluated by designing two experiments in three benchmarking scenarios. The first test is intended to determine how the planners perform a motion planning task similar to an actual screwing process of a radio filter. The purpose of the second experiment is to investigate how the planners perform as the density of the obstacles increase. The performances of the planners have been analyzed and compared with each other using different benchmarking tools such as the planner arena.

Result of this project indicates, KPIECE and EST can outperform the state-of-the-art planner, RRT-Connect in some metrics, especially in an environment with a low obstacles density. However, RRT-Connect is still superior in more dense and complicated settings.

Keywords

Robotic manipulation, Motion planning, OMPL, MoveIt, UR5, KPIECE, PDST, EST.

Sammanfattning

Rörelseplanering är en viktig komponent i ett automatiserat system. Detta projekt syftar till att designa en rörelseplaneringsmodul för att automatisera skruvningen av radioenheter. Målet är att implementera en rörelseplanerare som kan frambringa den hastighet, noggrannhet och effektivitet som krävs för en automatiserad skruvdragare. Skruvdragarens uppgift är att skruva ett antal hål placerade nära varandra på en radiofilter. Denna uppsats har undersökt fyra kontrollbaserade rörelseplanerare på en 6 Degrees Of Freedom (DOF) robotarm med hjälp av Robot Operating System (ROS). De undersökta rörelseplanerarna är Rapidly-Exploring Random Trees, The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration (KPIECE), The Path-Directed subdivision Trees (PDST) och Expansive Space Trees (EST) som är tillgängliga i Open Motion The Open Motion Planning Library (OMPL). Planerarna implementeras på en simulerad UR5-robotarm, genererad av MoveIt Setup Assistant, som är det primära verktyget för att skapa konfigurationsfiler för kinematikkedjor i MoveIt.

ROS är den valda plattformen för att designa styrmetoder och rörelseplaneringens algoritmer vilket medför att två arbetsytor har skapats. Dessa arbetsytor innehåller flera paket och noder med flera uppgifter bland annat rörelseplanering, visualisering och dataextraktion. Alla noder kommunicerar med varandra genom ROS kommunikationsverktyg liksom massagetjänster och action-client tjänster.

Detta projekt omfattar även benchmarking av alla ovannämnda planerare för att avgöra vilken av dessa planerare kan åstadkomma en optimal prestanda i olika miljöer. Planerarens prestanda utvärderas genom att designa två experiment i tre benchmarking-scenarier. Det första testet är avsett att bestämma hur en planerare utför en rörelseplaneringsuppgift vilket liknar en verklig skruvprocess för en radioenhet. Andra experimentet är att undersöka hur planerarna presterade när tätheten av hindren ökar. Planerarnas prestationer har analyserats och jämförts med varandra med hjälp av olika benchmarkingverktyg, till exempel Planer Arena.

Enligt resultatet av detta projekt kan KPIECE och EST prestera bättre jämfört med den senaste planeraren RRT-Connect i vissa områden, särskilt i ett miljö med låg hindertäthet. RRT-Connect är dock fortfarande överlägsen i mer täta och komplicerade miljöer.

Nyckelord

Robotmanipulation, Rörelseplanering, OMPL, MoveIt, UR5, KPIECE, PDST, EST.

Acknowledgments

I am deeply and sincerely grateful to my supervisors, Jörg Conradt and Sakari Salmi, and my co-supervisor Juan Romero, for their support and the valuable knowledge and experience they shared with me. I would also like to express my appreciation to my examiner Arvind Kumar. Completing this project would not be possible without the assistance of these people. Finally, I would like to thank my family for their love and support throughout my life, especially during my years of studies at KTH.

Stockholm, January 2022

Syedhesam Ahmadi

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	2
1.3	Research Question	2
1.4	Goals	2
1.5	Limitation	3
1.6	Ethic and sustainability	3
2	Background	5
2.1	Related work	5
2.2	Robot model	6
2.2.1	Degree of freedom of the UR5	7
2.3	Orientation	8
2.3.1	Unit quaternions	8
2.3.2	Representation of rotations and massage transformation	9
2.4	Motion planning	9
2.4.1	Sampling Theory	11
2.4.2	Kinematic configuration	11
2.5	Motion planning tools	13
2.5.1	The Robot Operating System	13
2.5.2	MoveIt!	13
2.5.3	The Open Motion Planning Library	14
2.6	Motion planners	14
2.6.1	Rapidly-exploring Random Trees	15
2.6.2	Expansive Space Trees	16
2.6.3	The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration	18
2.6.4	Path-Directed Subdivision Trees	20

3	Methods	23
3.1	Motion planning and visualization	23
3.1.1	Description of ROS packages and their design purposes	24
3.1.2	Description of nodes and their underlying structure	25
3.2	Defining kinematic, visual and collision boundary of the UR5	27
3.2.1	Kinematics chain generation	27
3.2.2	Define Object's geometry, visual and collision boundary	28
3.3	Algorithm benchmarking	29
3.3.1	Primary benchmarking	29
3.3.2	Second experiment	30
3.4	Data Collection	32
4	Results	33
4.1	Result of primary experiment	33
4.2	Result of second experiment	35
4.2.1	Planning time	35
4.2.2	Path length	37
4.2.3	Solution smoothness	39
4.2.4	Memory usage	41
4.2.5	Number of solved cases	42
4.2.6	Summary of the motion planners performance	44
5	Discussion	45
5.1	Implications of the results	45
5.2	Another conceivable method	46
5.3	Assessing reliability of the data	46
6	Conclusions and Future work	48
6.1	Summary	48
6.2	Conclusions	48
6.3	Future work	49
6.4	Reflections	50
	References	51
A	Essential concepts of ROS	55
B	Flexible Collision Library	57
C	KDL Inverse Kinematic solver	59

D	Move_group_interface	60
E	Rviz GUI	62
F	OMPL	63
G	Listing	66

List of Figures

2.1	The piano mover's problem.	10
3.1	Structure of ROS nodes and topics. The ellipses represent the nodes in this figure, and the boxes represent the topics.	26
3.2	Three benchmark scenarios with different number of obstacles designed to evaluate the performance of motion planners	30
3.3	A filter unit and with two different screw types for inner holes and outer holes	31
4.1	Average plan time achieved by the planners.	35
4.2	Plan time obtained by the planners in the secondary experiment.	36
4.3	Average path length achieved by the planners.	37
4.4	Length of the path obtained by the planners in the secondary experiment.	38
4.5	Average Solution smoothness achieved up by the planners.	39
4.6	The smoothness of the solutions obtained by the planners in the secondary experiment.	40
4.7	Average memory taken up by each planner.	42
4.8	The relative performance of the motion planners.	44
B.1	FCL structure	58
F.1	OMPL structure.	63

List of Tables

2.1	Specification of the UR5 joints	7
4.1	Average planning time and execution time obtained by the planners in the primary experiment	33
4.2	The metric accuracy obtained by the planners in the primary experiment	34
4.3	Number of solved cases for each planner in the first test scenario of the second experiment.	43
4.4	Number of solved cases for each planner in the second test scenario of the second experiment.	43
4.5	Number of solved cases for each planner in the third test scenario of the second experiment.	43

Listings

D.1	Motion planning using Move_group interface	61
G.1	Message format uses for transferring the sensor's information .	66
G.2	Example of a child-parents relationship between the table and the robot.	66
G.3	Myworkcell file which contains information about various planning groups joints and their types	67
G.4	A snippet describing how to assign visual and geometry properties to a link and also how define a connection between two links via a joint.	67
G.5	Planning time	68
G.6	Metric accuracy	68
G.7	Calculation of planning accuracy.	69
G.8	A .yaml configuration file that contains information about the scenes and queries	69
G.9	tf2::Quaternion object which can be used to transform Euler representation to quaternion representation.	69

List of acronyms and abbreviations

CCD Continuous Collision Detection

DOF Degrees Of Freedom

EST Expansive Space Trees

FCL Flexible Collision Library

GUI Graphical User Interface

IK Inverse Kinematics

KDL Kinematics and Dynamics Library

KPIECE The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration

NR Newton-Raphson

OMPL The Open Motion Planning Library

PDST The Path-Directed subdivision Trees

ROS Robot Operating System

RRT Rapidly-exploring Random Trees

SRDF Semantic Robot Description Format

SVD Singular Value Decomposition

URDF Unified Robot Description Format

Chapter 1

Introduction

1.1 Background

Today's competitive industry puts increasing demands on production efficiency, especially on early-stage prototyping. Test and system modification in the prototyping requires assembling and disassembling of prototype units. This process is usually time-consuming and repetitive.

Ericsson is one of the companies that is leading the telecommunication market by continuously introducing new products. However, developing new products at a rapid pace requires an even more durable and agile prototyping methods. Hence, Ericsson continuously searching for reliable, fast and automated solutions for the prototyping process so that engineers can focus on design rather than assembly. This project is a major part of developing an autonomous screwdriver and aims to design a motion planner for a robot arm that will automate the screwing process of radio filter prototypes.

Motion planners are an essential part of every utterly automated system. A motion planner receives a description of the environment and a specific task as input and produces a motion. Various motion planning algorithms provide different advantages depending on the job and the environment. Hence, it is imperative to define a specific problem and environment to evaluate motion planners' performance accurately.

In this project, our task requires a motion planner that performs the planning process regardless of the filter model and shape in the presence of obstacles. Furthermore, The motion planner is expected to provide speed and precision without sacrificing quality and accuracy. The specification of the problem will be discussed in more detail in Section 1.2.

1.2 Problem

This project investigates and compares various motion planners to design an optimal motion planning process suitable for a 3D workspace in the presence of static obstacles. This project studies different control and sample-based motion planners and compares their performances with the state-of-the-art motion planner RRT-Connect. The motion planner requires to minimize execution time, planning time, and memory usage while providing a smooth path with a minimized length.

Furthermore, a high level of accuracy for motion planners is also required since the outcome of this project will be used to develop an autonomous screwdriver that requires precise positioning and movement of the end-effector.

1.3 Research Question

The question that has been investigated in this project is: To what extent can [The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration \(KPIECE\)](#), [Expansive Space Trees \(EST\)](#), and [The Path-Directed subdivision Trees \(PDST\)](#) motion planners outperform the state-of-the-art planner, RRT-Connect in terms of path length, planning time, execution time, solution smoothness, and accuracy, given that the motion planners are implemented on a robot arm with six DOF in a three-dimensional that must avoid any collision with obstacles and environmental object.

The motion planners have been studied in various workspaces, and the performance of the motion planners has been evaluated based on the environment specification and especially the number of obstacles. Thus, this project also aims to provide an answer to the following questions:

Which planners will provide the best performance in a specific environment in terms of the metrics mentioned above, and which motion planners will provide the best overall performance and possibly outperforms RRT-Connect.

1.4 Goals

The primary goal of this project is to evaluate the performance of various sample-based motion planners in different environments. The benchmark setups have been designed thoughtfully to represent realistic and challenging situations and make the comparison with state-of-the-art planner significant. The other goal of this project is to design a functional motion planning process for an autonomous screwdriver that can perform the following tasks.

- Plan and execute a path from a screw feeder to a coordinate on the filter radio surface where a screw hole is located.
- Plan and execute a path back to the screw feeder.
- Repeat this process until the end-effector covers positions of all the screw holes.
- Be able to detect and avoid all the static obstacles and re-plan the path if there is a risk for collision.

The intention is to choose a planner that achieves these goals while fulfilling the requirements mentioned in the problem section and providing reliable and robust performance.

1.5 Limitation

This project is intended to include both simulations and also the implementation of motion planning algorithms on an actual robot. But due to time constraints and the complexity of this project, implementing motion planners on an actual robot arm was excluded from the project in later stages. However, all of the project's results will be evaluated based on concrete and comparable simulations so that the results and findings will be scientifically valid.

Furthermore, grasping and screwing of the screws are excluded from this project as they are irrelevant to this thesis's primary purpose and don't add any scientific value to this project.

1.6 Ethic and sustainability

This project was conceived to reduce the test and system modification's inefficiency and difficulties in early-stage prototyping, which requires assembling and disassembling prototype units. The assembling process is usually time-consuming and repetitive.

This project provides methods with clear advantages and disadvantages that can automate the screwing process for radio units which is extra beneficial in early-stage prototyping. In early-stage prototyping for the radio units, the location of the screw varies often based on the prototypes model. Thus, designing a production line for each prototype version is not economical and performing it in a traditional manual fashion is also time-consuming and inefficient.

The alternative solution is to design an autonomous system that performs the

screwing process without any prior knowledge about the filter model and the screwing pattern. This project presents a wide range of motion planners that can plan and traverse the robot arm between screw locations in real-time while avoiding environmental obstacles. The motion planning module is a backbone for an autonomous screwdriver that performs the screwing task online and can be considered as a reliable and sustainable alternative for the manual way of performing assembly and disassembly tasks.

Another benefit of an autonomous screwdriver is eliminating the need for the operator to do the screwing task manually. The screwing process is a heavy task that can lead to physical injury, especially if the human operator does it for many years. By adopting this technology, there is no need for a human operator. Instead, the machine does this repetitive task that decreases the human operator's mental or physical exertion and fatigue.

Chapter 2

Background

This chapter introduces the robot model and the calculation of DOF of the system and a short introduction to unit quaternions. Section 2.4 introduces the underlying concept of motion planning, kinematic configuration and sampling theory. Furthermore, the concepts of [Robot Operating System \(ROS\)](#), [MoveIt](#) and [The Open Motion Planning Library \(OMPL\)](#) are explained in respective subsections 2.5.1, 2.5.2 and 2.5.3. Finally, section 2.6 presents the underlying theory of all the motion planners investigated in this project.

2.1 Related work

Motion planning is one of the most discussed topics in robotics. There are various methods for motion planning. One recent example of novel motion planners is the neural network-based planning algorithm. This network is known as MPNet [1]. MPNet comprises two neural networks, an encoder network (Enet) and a planning network (Pnet). Enet takes the environment information such as a point cloud converted to a voxel. Pnet takes the encoding of the environment, the robot's current state, and goal state to output samples for a path. This method utilizes a two algorithm known as Gradient episodic memory for continuum learning to solve the problem of catastrophic forgetting and solve various problems without forgetting previous tasks [2].

Another method for planning the motion for complex systems is sampling-based motion planning which has become more widespread recently. All sample-based motion planning, sample the environment either by point checking or generating segments and finds a path between the start and goal state irrespective of obstacles geometry if a feasible path exists. However, these methods have their weaknesses. They fail to find the absolute shortest path solution in a finite time, and they become computationally inefficient as the

dimensionality of the planning problem increases. There are a few exceptions that can find the optimal path, such as RRT* [3] and PRM* [4]. These methods are optimizing planners and provide optimality guarantees with the cost of having a slower plan time.

Another family of motion planning algorithms are multi-query planners. Probabilistic roadmap is the most utilized multi-query planner, presented in [5]. This method takes a new approach to explore the environment. This algorithm builds and retains a roadmap in the workspace that can be reused for later planning attempts. The main advantage of using PRM is that it keeps a structure of the generated graph to reuse it for every query, and the motion planner does not require to create a graph in each run.

On the other hand, single query motion planners typically build one or two trees of connected nodes by exploring the environment. In [6], a single-query planner is presented. This method is known as Kinematic Planning by Interior-Exterior Cell Exploration and suitable for operations with complex dynamics. In this method, the workspace discretizes into a grid used to estimate the state space coverage. The exploration strategy is biased towards the unexplored areas of the state space. So, this method keeps track of the explored region with a specific boundary such that the majority of the exploration takes place outside of this boundary using state space coverage. KPIECE does not require any state sampler and is a desirable motion planner for problems characterized by its physical models.

Finally, control-based planning methods are a suitable option if system is subject to differential constraints. Control-based motion algorithms derive differential equations from the system's motion using various methods and imply control theory concepts such as feedback and feed-forward to follow the trajectory. In [7], A control-based method called Expansive Space Trees is presented. This method is a tree-structured motion planner that explores the state space biased towards the less explored areas. EST is a bidirectional motion planner similar to RRT-Connect. Hence, it grows two trees, rooted in a start and a goal state. The EST's main advantage is the ability to handle difficulties caused by the presence of narrow passages that prevent many planners from reaching to the goal state, especially if the start and goal states can only be connected to each other via a narrow passage.

2.2 Robot model

UR5 is a relatively adaptable, lightweight, and collaborative robot produced by Universal Robots and is a member of the CB3 family. UR5 is a medium-sized industrial robot ideal for automating different duties with low-weight

and medium-duty applications with high accuracy and flexibility. This robot is easy to set up and grants a perfect ratio between the generated power and its weight. UR5 has six rotational joints with the specification presented in Table 2.1.

Joints	Motion Speed	Joint range
Wrist 3	180°/s	± 360
Wrist 2	180°/s	± 360
Wrist 1	180°/s	± 360
Elbow	180°/s	± 360
Shoulder	180°/s	± 360
Base	180°/s	± 360

Table 2.1: Specification of the UR5 joints

2.2.1 Degree of freedom of the UR5

The **DOF** for a robot arm is the dimension of the configuration space (c-space), which in turn is the minimum real number needed to represent the c-space of that robot. According to Grubler's formula, degrees of freedom for a mechanism are derived by subtracting a joint's freedom from the number of independent constraints. [8].

$$\text{DOF} = \sum (\text{Freedoms of bodies}) - \text{Number of independent constraints} \quad (2.1)$$

For a robot arm with J number of joints this formula become :

$$\text{DOF} = m(N - 1) - \sum_{i=1}^j c_i \quad (2.2)$$

Where m is the number of freedoms for a single rigid body, for spatial bodies, m equals six with three rotational and three transitional in a 3D space. For planer bodies, m equals three with one rotational and two transitional in 2D space. In our case, the planning takes place in a 3D space that sets m equal to six. N is the number of bodies inclusive of the ground. UR5 has six revolute joints, and adding ground to this number sets N equals seven. Finally, c is the constraint between two rigid bodies. For a revolute joint, c is equal to 5. Inserting these values in Equation 2.3 gives the DOF of the UR5 robot arm.

$$\text{DOF} = 6(7 - 1) - 6(5) = 6 \quad (2.3)$$

2.3 Orientation

There are several options to describe spatial rotations in 3D space. The two most common ways of representing rotation are unit quaternions and Euler angles. This section will describe both representations briefly and how to convert Euler angles to unit quaternions [9].

2.3.1 Unit quaternions

There are several ways of describing spatial rotations in a three-dimensional space. One of the most common ways of representing rotation is the unit quaternions, also known as versors. The main idea of quaternions is to add one extra dimension to multiply triples. A quaternion q is a summation of a scalar and a three-dimensional vector. For a scalar q_0 and a vector $q = [q_1, q_2, q_3]$, the quaternion q is defined as :

$$q = q_0 + \mathbf{q} = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}. \quad (2.4)$$

q consist of a scalar part q_0 and a imaginary vector $\mathbf{q} = q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$. The imaginary unit vectors i, j and k have the following relationship [10]:

$$\mathbf{i}^2 + \mathbf{j}^2 + \mathbf{k}^2 = \mathbf{ijk} = -1 \quad (2.5)$$

Unit quaternion is a quaternion with a norm equal to 1. The norm of a quaternion is a scalar property and derives from $|q| = \sqrt{qq^*}$. In this equation, q^* star is the complex conjugate of q . The conjugate of q derives by reversing the sign of the imaginary part of vector $\mathbf{q}$. Finally, the inverse of a quaternion q is defined as :

$$q^{-1} = \frac{q^*}{\|q\|^2} \quad (2.6)$$

With the inverse quaternion defined, it is possible to verify if q is a unit quaternion or not. By definition, q is a unit quaternion if $q^{-1}q = qq^{-1} = 1$ which is equivalent with, the inverse of q be same as its complex conjugate. As mentioned earlier, a unit quaternion q can be written as the sum of an imaginary part and a real part, namely, $q = \text{Im}g(q) + \text{Real}(q) = q_0 + \mathbf{q}$ while $\|q\| = 1$ [10]. Absolute value equals to one, implies that $q_0^2 + |\mathbf{q}|^2 = 1$. Comparing this equation with the "Pythagorean" identities, we can conclude that $\sin(\phi)^2 = |\mathbf{q}|^2$ and $\cos(\phi)^2 = q_0^2$. In fact, $\cos(\phi) = q_0$ and $\sin(\phi) = \|q\|$ for a specific angle $\phi \in [0, 2\pi]$ and the unit quaternion q , can be formulates in terms of this angle and a unit vector u as following:

$$q = \cos(\phi) + u \sin(\phi), u = \frac{q}{\|q\|} \quad (2.7)$$

2.3.2 Representation of rotations and message transformation

ROS uses quaternions to represent rotations in $\mathbb{R}^3$. In ROS a quaternion q is usually described as (x, y, z, w) while (x, y, z) is the real part of q and w is the scalar part denoted by q_0 .

It is challenging to imagine rotations in terms of quaternions as it is difficult to describe elements of quaternion with physical phenomena. For this reason, rotations have been implemented in terms of Euler angles and the result converted to a quaternion afterward. A `tf2::Quaternion` transformation object is created to perform this transformation which receives the rotations in terms of roll, pitch and yaw and returns the rotation in terms of unit quaternions. (See Listing G.9 for representation of the `tf2::Quaternion` object).

The alternative way of injecting rotations is to pass the Euler angles directly to the kinematic solver. **Kinematics and Dynamics Library (KDL)** (see Appendix C) kinematic solver can receive and return Euler angles directly. By invoking the constructor of the `KDL::Rotation` class and calling the `Rotation::RPY(double roll, double pitch, double yaw)` function, angles in Euler format can be injected into the kinematic solver.

2.4 Motion planning

The objective of a motion planning process is to find a solution to the problem of moving from an initial state to a goal state in the state space while satisfying the constraints forced by either the robot limitations or the obstacles in the environment. A famous example of motion planning is the classical piano mover's problem. This problem is described as finding a path for a rigid body, piano, which starts from one initial state and ends at a goal state. Figure 2.1 [11] presents a scenario of moving a piano in the environment. In this problem, all the obstacle's locations are assumed to be known, and the main challenge is to find a collision-free path connecting the start and end states. A rigid body such as a piano in 3D space has six degrees of freedom, including 3 DOF for movement in the x , y and z planes, and three more named rolls, pitch, and yaw to represent rotation along the axes of these planes. In this setup with a high degree of freedom, finding an exact solution is challenging. To find a collision-free path, the motion planner needs to compute continuous changes for the set of $(x, y, z, \text{roll}, \text{pitch}, \text{yaw})$. From a mathematical point of view, motion planning problem is defined as finding a set of control inputs $u : [0, T] \rightarrow u$ for a given initial state X_{start} and a desired final state X_{final} in a specific amount of time T such that the motion of the robot along the trajectory satisfies $x(T) =$

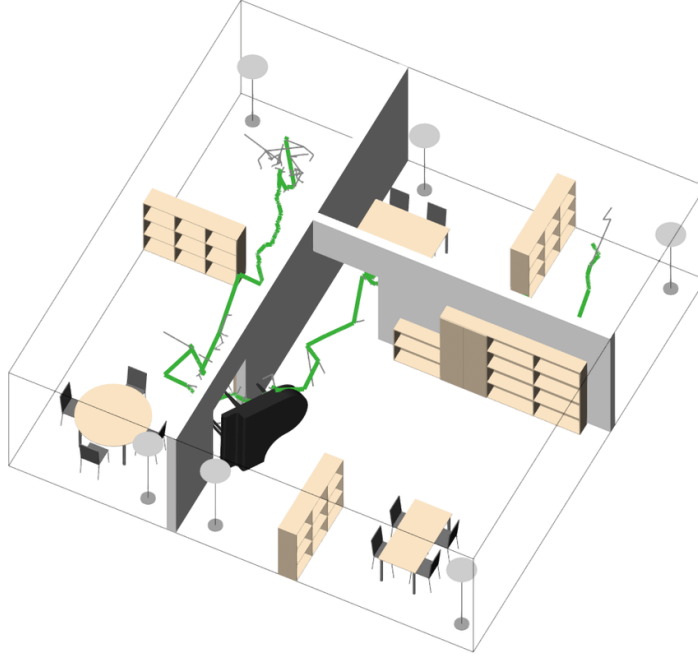


Figure 2.1: The piano mover's problem.

x_{goal} and $q(t)$ in C_{free} for all $t \in [0, T]$. For example, motion of the robot $\dot{x}$ with configuration $q \in C \subseteq R^n$ and $C = C_{free} \cup C_{obs}$ and acceleration as the a control inputs where $(q, v) \in X$, can be written as Equation 2.8.

$$\dot{x} = f(x, u), u \in U \quad (2.8)$$

Where u is an element of the feasible control set U .

$$x(T) = x(0) + \int_0^T f(x(t), u(t)) dt \quad (2.9)$$

Single-query and multi-query motion planner

A Motion planner can be either a multi-query or single-query. A multi-query planner invests time in generating a good representation of the environment so that future motion planning problems in the same space can be solved quickly. On the other hand, a single query planner attempts to solve a single motion planning problem as quickly as possible by finding a connected path between the states in each run separately from the other planning attempts.

Completeness

A motion planner is complete if it always finds a valid solution for a motion planning problem if one exists. On the other hand, a motion planner has resolution completeness if the planner finds a solution whenever a solution exists. Finally, a motion planner is probabilistically complete if the probability of finding a solution, if one exists, is 100 percent if the planning time tends to infinity [12].

2.4.1 Sampling Theory

The state space for motion planning in real-world scenarios is either a two-dimensional area or three-dimensional space med infinite amounts of states, which significantly limits the exploring and planning processes. Hence the main idea of a sampling-based algorithm is to address this limitation. A sampling-based planning algorithm can consider a countable number of samples. However, if the algorithm runs forever, the number of states becomes countably infinite. In practice, a motion planning process utilized with sample-based algorithms is expected to terminate after a finite number of iterations (samples).

2.4.2 Kinematic configuration

Inverse Kinematics (IK) is defined as finding a solution to the problem of determining a value set for the joints(orientation and position) as the end effector is in specific coordination in the cartesian space. In common motion planning problems, trajectories are defined in cartesian space and not in configuration space. Hence, the **IK** methods can be utilized to perform this transformation and provide a set of joint configurations given the pose of the end effector.

The kinematic equations are used to set up closed-loop equations with non-linear constraints on the configuration variables of the robot's joints. The number of constraints on the configuration variables of a closed-loop system equation is also known as the **DOF** of that system. For many kinematic equations, there is no guarantee for an **IK** problem to have an analytical solution. For instance, if the **DOF** of a robot exceeds the **DOF** of the end-effector, infinite amount of solutions for the **IK** problem exists. Hence this problem lacks an analytical closed-loop solution.

The closed-loop solution has a significantly high computational complexity. Therefore the computation times would be substantially higher than the numerical techniques. The numerical solutions are not as accurate as the

analytical method. However, the accuracy of numerical techniques is often sufficient for most motion planning problems.

Derive IK equation

Consider a kinematic system with n number of joints where each cartesian coordinate is a function of the joint angles. Let $X = [x, y, z, \phi_1, \phi_2, \phi_3]$ be the cartesian coordinates (pose) and configuration $\Theta = [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6]$ be the joint angles. Then the forward kinematic K_f is defined as:

$$X = K_f(\theta) \quad (2.10)$$

The differential kinematics are obtained by taking the partial derivatives of each Cartesian coordinate with respect to each joint angle which is shown in Equation 2.11.

$$\dot{X} = J(\theta)\dot{\theta} \quad (2.11)$$

The Jacobian $J(\theta)$ is the Jacobian of the K_f describing the transformation between velocities in configuration or cartesian spaces where each element j_{mn} is derived from $\frac{\partial K(\theta_m)}{\partial \theta_n}$. Thus, each column in J can be seen as the motion in X (ΔX_i) caused by motion in the joint θ_i . The number of rows of the Jacobian matrix equals the DOF of the robot, and the number of columns is equal to the number of joints of the manipulator.

Configuration θ is the solution of Equation 2.11. One method to solve this equation system is to define a linear least squares optimization problem that minimizes $\|\dot{X} - J(\theta)\dot{\theta}\|$. The optimal fit can be determined through an n -dimensional hyperspace and solved by a gradient descent method. A famous example of a gradient descent method is the **Newton-Raphson (NR)** method [13] which offers one such form of minimization. Multiplying of both side of Equation 2.11 to the inverse of Jacobian, namely $J(\theta)^{-1}$ returns the representation of joint velocities in terms of cartesian velocities and inverse Jacobian which is shown in Equation 2.12.

$$\dot{\theta} = J(\theta)^{-1}\dot{X} \quad (2.12)$$

The inverse of the Jacobian matrix is trivial to calculate if this matrix is invertible. Inverse Jacobian, $J(\theta)$ can be derived only for a square matrix. The Jacobian of a robotic system is a square matrix if and only if the number of joints is equal to the DOF of the robot. If J is not invertible, pseudo-inverse can be used instead. Different pseudo-inverse methods can be utilized to calculate the inverse of the Jacobian for redundant arms approximately. Another option is to use **Singular Value Decomposition (SVD)** method to derive the pseudo-inverse of the Jacobian[14].

2.5 Motion planning tools

2.5.1 The Robot Operating System

ROS ¹ is a flexible and open-ended collaboration framework to develop robotic software. ROS contains a set of tools, libraries and infrastructure that aims to ease the task of developing complex and robust automated practices across a broad spectrum of robotic applications. The ROS community consists of many worldwide users who work on projects of varying scales and complexity for research and industrial purposes (see Appendix A for more information about the ROS and the communication tools ROS provides).

2.5.2 MoveIt!

MoveIt! ² is one the most comprehensive open-source software for robotic manipulation. MoveIt is compatible with a large variant of robots and has been considered the state-of-the-art tool to solve various robotic problems such as manipulation, navigation, control, etc. Moreover, MoveIt provides a user-friendly ecosystem for a developer to perform:

1. Implementation of the complex and advanced robotics applications.
2. Simulation and visualization of different environment instances such as obstacle, point of interest and the robot.
3. Evaluation of motion planning results.

MoveIt provides all the necessary functionality needed for robotic manipulation. The property that makes MoveIt a valuable and unique tool is a vast library of robotic capabilities for manipulation, motion planning and control available for MoveIt users.

MoveIt! Structure

MoveIt has been designed as a plugin-based structure. Different packages and services are connected to a central node called move group. Move group is a collection of classes, structs, unions and interfaces that provide essential functionality for motion planning, moving the robot, adding objects and obstacles into the environment. MoveIt is specially adapted for the ROS environment to enable users and developers to improve and extend MoveIt capabilities [15].

¹<https://www.ros.org/>

²<https://moveit.ros.org/>

MoveIt! APIs

MoveIt supports various Interfaces, enabling users to access the functionalities and services provided by the Move group. The three primary Interfaces are `Move_group_interface`, `MoveIt_commander` and Rviz GUI. `Move_group_interface` (see Appendix D) and Rviz GUI (see Appendix E) are the two interfaces used in this project.

2.5.3 The Open Motion Planning Library

OMPL is a collection of state-of-the-art sampling-based motion planning algorithms, including implementations of a large variety of sample-based planning algorithms. OMPL represent all the essential concept of motion planning such as state space, workspace, goal and start state with an abstract representation. Hence it is a suitable library to solve and evaluate both simulation and physical robotic problems.

Note that OMPL does not include the geometry representation of the workspace or the robot. Consequently, the user must define the kinematic configuration of the robot and pick a suitable collision detection method. Different tools and formats exist to represent the robot kinematics and collision boundaries, such as [Unified Robot Description Format \(URDF\)](#) files. The user must select the collision checking module since OMPL does not have any default collision checker. However, it is compatible with several collision checking methods such as [Flexible Collision Library \(FCL\)](#) (see Appendix B) [16].

OMPL consists of several modules to execute the motion plannings process. These modules have specific tasks such as the sampler module which is responsible for computing valid configurations, the state validity checker module evaluates the robot configuration's validity, and the local planner module finds a local collision-free path between adjacent samples (See Appendix F for more information about the modular structure of OMPL).

2.6 Motion planners

Control-based planners are a viable option when a system is subject to differential constraints. The goal is to navigate a robot in continuous space along the desired trajectory, either offline or online. Control-based motion algorithms derive differential equations from the system's motion using various modeling methods and imply control theory concepts such as feedback and feed-forward to follow the trajectory with minimal bias. In addition, these planners use the state propagation library to generate motions between states without requiring the user to include any specific steering module. Subsection

2.6.1, 2.6.2, 2.6.3, and 2.6.4 present the fundamental theories of the investigated planners in this paper.

2.6.1 Rapidly-exploring Random Trees

Rapidly-exploring Random Trees (RRT) is an algorithm developed by Steven M. LaValle and James J. Kuffner Jr. RRT aims to solve the problems containing nonholonomic constraints since many of the current motion planners, such as randomized potential fields or probabilistic roadmap, can not be generalized to the nonholonomic problems [17]. RRT inherits some of the useful properties from its processors in combination with new distinctive properties. For example, RRT does not require each pair of states to be connected, which provides practical advantages. As a result, it can solve a broader class of problems and be applied to kinodynamics path planning. [18]. RRT, similar to the other path planner, tries to connect the start state x_{init} to the goal state x_{goal} by searching the environment and avoiding the static obstacles x_{obs} . However, an explicit representation of the obstacles is not available, so the RRT searches the environment by generating new states such that all the vertices and edges lie in the free space x_{free} . In this definition, x_{free} is the collection of all the states which do not include any obstacle, and edges represent the path between states belonging to the tree [19]. As mentioned earlier, this process includes nonholonomic constraints. These constraints are expressed as a state transition equation $\dot{X} = f(X, U)$, where U is the inputs and X represents the state vector. RRT algorithm for an initial state x_{init} with K vertices for a general configuration space, $\mathcal{C}$ is constructed according to Algorithm 1³.

Algorithm 1 RRT Algorithm

```

Generate_RRT( $x_{init}, K, \Delta_t$ )
 $G_{init}(t)(x_{init}());$ 
for  $i \leftarrow 1$  to  $K$  do
     $x_{rand} \leftarrow \text{Random\_State}();$ 
     $x_{near} \leftarrow \text{NEAREST\_Neighbor}(x_{rand}, G);$ 
     $u \leftarrow \text{Select\_Input}(x_{rand}, x_{near});$ 
     $x_{q_{new}} \leftarrow \text{New\_State}(x_{near}, u, \delta_t);$ 
     $G.\text{add\_vertex}(x_{new});$ 
     $G.\text{add\_edge}(x_{near}, x_{new}, u);$ 
    return  $G$ 
end for

```

³<http://msl.cs.uiuc.edu/rrt/about.html>, web page maintained by Steve LaValle

In this algorithm, The `Random_State()` function choose a random state in the state space and the $NEAREST_Neighbor(x_{rand}, G)$, selects the vertex, x_{near} , in G (set of the state belonging to the constructed tree), that is closest to x_{rand} . The implementation of this part is shown in Algorithm 2.

Algorithm 2 $NEAREST_Neighbor$ function

```

 $d \leftarrow \infty;$ 
for  $v \in V$  do
  if  $\rho(q, v) < d$  then
     $v_{new} = v; d \leftarrow \rho(q, v);$ 
    Return  $q;$ 
  end if
end for

```

Whenever a sample is generated, the algorithm attempt to connect it to the nearest state from the set of states belonging to the tree. These newly generated samples add to the tree structure if a connection is possible, and a path between the new state and the nearest state lies entirely in the free space.

RRT algorithm has been modified with new proprieties. So today, different versions of the RRT algorithm have been constructed with various use cases. RRT-Connect and RRT star are two famous examples of such algorithms. RRT-Connect is one of the motion planners investigated in this project. The difference between RRT and RRT-Connect is that RRT-Connect expands two trees from two fronts, the start and the goal state. This modification makes the algorithm more biased towards the goal target, and the search will be more goal-oriented. RRT star is the optimal version of RRT that grantees to return the best path from the initial state to the goal state as time goes to infinity.

2.6.2 Expansive Space Trees

EST [20] is a sample-based and tree-structured motion planner that explores the state space biased towards the less explored areas. It is necessary to understand some of the underling concepts of EST before diving into the EST algorithm.

Configuration space C is defined as the set of all the reachable Configuration for a robot. A configuration can either be free or occupied. A configuration q is free if the robot pose in this configuration has no collision with the obstacles. The set of all free configurations in configuration space is called free space, and the set of free configurations retained in a roadmap is called milestones. Suppose two configurations can be connected to each other with a straight line that entirely lies in the free space. In that case, these two configurations

are able to see each other and which means they are in each other's visibility region.

The EST's main advantage is the ability to handle difficulties caused by the presence of narrow passages in the workspace that prevent many planners from creating a roadmap. These narrow passages divide the configuration space into different separate subsets. Hence, if the start and goal states happen to be in different subsets, these two configurations can not see each other and making the planning process significantly more complex [21].

Algorithm

EST is a bidirectional motion planner similar to RRT-Connect. Hence the algorithm simultaneously creates two trees rooted in a start and a goal state. The planner samples the configuration space on each side and keeps the nodes that see their prior nodes. Consequently, this node can be connected to the prior nodes by a path. Note that each tree has its own visibility region formed by the expansion of the visibility regions of the nodes belonging to that tree. The sampling process repeats until the visibility regions of the two trees intersect with each other.

The following pseudo-codes present the EST algorithm. This algorithm consists of two parts, expansion and connection.

The expansion algorithm, which is described in Algorithm 3 creates two trees, one rooted in the start state and the other in the goal state. In the beginning, a node called x initializes with an initial probability. This probability $prb(x)$ derives from $prb(x) = \frac{1}{w(x)}$. In this formulation, $w(x)$ is the weight of the node x and the value 1 is the weight of the entire configuration space. Then, K samples are selected in the neighborhood of node x in a specific distance d . Afterward, the points with the highest possibility of extending the visibility region of the tree are retained. Finally, the algorithm checks if there exists a straight line between the points in the neighborhood of x and points in the neighborhood of y .

The connection algorithm responsibly is to create a path between two trees, T_{init} and T_{goal} , which is presented with details in algorithm 4.

Algorithm 3 Expansion Algorithm

Pick a node x from the free configuration set V with probability $prb(x)$.
 Sample K points from $N_d(x) = \{q \in C | dist_c(q, x) < d\}$.
for y in the picked from the configuration set **do**
 Calculate $w(y)$ and retain with probability $prb(y)$.
 if y is retained, and $link(x, y)$ return Yes **then**
 Put y in V (free configuration set), place an edge between x and y .
 end if
end for

Algorithm 4 Connection Algorithm

for Every $x \in V_{init}$ and $y \in V_{goal}$ **do**
 Calculate $w(y)$ and retain with probability $\frac{1}{w(y)}$.
 if $dist_w(x, y) < l$ **then**
 $link(x, y)$
 end if
end for

2.6.3 The Kinodynamic Motion Planning by Interior-Exterior Cell Exploration

KPIECE [22] is a control-based motion planner suitable for operations with complex dynamics. The workspace discretizes into a grid used for estimating the state space coverage in this method. The exploration strategy is biased towards the unexplored areas of the state space. Hence, this method defines an explored region with a specific boundary such that the majority of the exploration takes place outside of this boundary using state space coverage. The key benefit of KPIECE is the computational improvement this method provides compared with the other state-of-the-art methods by having a shared-memory implementation and the ability of parallel computation. The additional benefit of this method is that KPIECE does not require any state sampler or distance metric modules. Hence, KPIECE is a desirable motion planner for problems characterized by its physical models. Moreover, sampling-based motion planners spend most of their computation capacity to create a searching tree, which is a disadvantage. In contrast, KPIECE takes less use of the state propagation module, which makes the algorithm faster and less computationally demanding. [6].

Algorithm

A motion planning problem formulates as a set of (Q, U, I, F, f) where Q is the set of all the possible states in the state space and U is the set of all possible states in the control space. Finally, I is a subset of Q and donates the set of all the initial states, and F is a subset Q and donates the set of all final states. A solution to this problem defines by a set of control sequences $\{u_1, u_2, \dots, u_n\} \in U$ in the time domain such that q_0 in I , q_n in F and q_k for $k = 1, 2, \dots, n$ can be obtained sequentially by integration of f .

KPIECE continually creates a tree of motions in the robot's state space. KPIECE, similar to RRT, constructs a tree for the motions in the state space. Each motion donates as $\mu = (s; u; t)$ where s is a state that belongs to configuration space, u is a control input belongs to the inputs vector U , and t donates the time when the control u is applied to initial state s and a motion is generated.

The algorithm aims to achieve the maximum coverage of the free space. So an indicator that estimates the coverage of Q is required. A discretized grid is also employed to guide the exploration towards the less covered area, and the motion tree grows in that direction.

After the discretization process, the cells are split into interior and exterior cells. Cells with $2n$ neighboring cells are considered interior, while cells with less than $2n$ neighboring cells are considered exterior. In this formulation, n is the dimension of projection of the state space, $E(Q)$. The tree of motion expands into new cells during the runtime, and several exterior cells will simultaneously become interior. Consequently, most cells will become interior when large parts of the state space are explored. This process is repeated until the goal is reached or the stopping condition is fulfilled.

Algorithm 5 [6] describes different steps of the KPIECE planning process. Here, P_j reflects the progress at discretization level j . The values α and β have to choose such that their addition, P_j is positive. If P_j is around one or greater, it signifies good progress.

Algorithm 5 KPIECE algorithm

```

 $\mu_0$  contains  $q_{start}$ 
Create G, a data-structure Grid
AddMotion( $\mu_0$ )
for  $i \leftarrow 1$  to  $N_{iterations}$  do
    Select a cell chain c from G, with a bias on exterior cells;
    Select  $\mu$  from c according to a half normal distribution
    Select s along  $\mu$ 
    Sample random control  $u \in U$  and simulation time t
    check if any motion  $(s, t, t_0)$  is valid
    if a motion is found then
        Construct the valid motion  $\mu_0 = (s, u, t_0)$  with max  $t_0$ 
        If  $\mu_0$  reaches the goal region, Return path to  $\mu_0$ 
        G.AddMotion( $\mu_0$ )
    end if
    for every level of discretization grid do
         $P_j = \alpha + \beta$ 
        Multiply the score of cell  $p_j$  in c by  $P_j$  if and only if  $P_j < 1$ 
    end for
end for

```

2.6.4 Path-Directed Subdivision Trees

PDST is a sample-based motion planner. This planner builds a tree similar to its predecessors but has a different scheme for exploring the state space and representing the environment's samples. Its primary design philosophy is to overcome difficulties in motion planning for robots with non-trivial dynamics and motion planning for systems with second-order dynamics. However, PDST maintains many features that belong to other sample-based planners, such as using a low-cost forward propagation and constructing a tree in state space by switching between the state selection and the propagation. In this method, samples are in the form of path segments instead of the states, and the exploration strategy is greedy and deterministic.

PDST decrease the number of samples taken from the workspace by using segments instead of states and removes the necessity for check-pointing the environment. Another distinct property of this planner is the exploration process. A wide range of sample-based planners select the samples completely uniformly, while in this method, the priorities for samples are weighted. Hence, each sample initializes with the current iteration number as its priority, and the priority value for samples will be doubled each time they are selected.

The PDST algorithm

In order to cover the complete PDST algorithm, some terms and concepts need to be explained first. In PDST terminology, a path is referred to as a non-negative mass. In other words, a mass is a collision-free trajectory that starts and ends in a certain time period T . The specific path from the root of the tree to each state is also contained in the mass. In this formulation, a collision-free path π from the $root(M)$ to a state q donates as $path(M, q)$ [23].

The propagation function extends the tree by adding new masses randomly to the existing tree. Propagation consists of two parts in this method, choose action and generate action. The choose action selects random states from the mass set and returns it to the generate operation. This operation returns a associated with a particular mass along its path. For instance, for mass $M = \pi$ with the time-span T , the choose function select a time $t \in T$ randomly and return $\pi(t)$. The generate operation creates a new mass by implying a random control signal starting from the chosen state. For example, if q is the output of the choose operation, the generate function, $generate(M, q)$ for the mass M , selects a control function u from the set of control functions U , and returns a new path π' and the new mass M' will be created from π' .

For the exploration, the algorithm selects a sample with the lowest priority. PDST keeps a set of cells that represent the entire state space Q . After each iteration, each cell with its samples is subdivided, and the priority associated with the samples will be updated. Thus, updating a cell provides two cells whose union is the original set. Measuring the cells' volume requires a function $\mu(C)$ to be defined. $\mu(c)$ is a normalized measure of the volume of a cell. By this definition the volume of state space $\mu(Q) = 1$ [23]. Algorithm 6 [24] describes the PDST motion planning process.

Algorithm 6 PDST algorithm

```

Set the subdivision to  $S = \{Q\}$  and create a single mass  $M_0 = \{q_0\}$ .
Add the mass  $M_0$  to the mass set  $\mathbf{M}$  and Set the  $priority(M_0 = 0)$ .
for  $i \leftarrow 1$  to  $N$  do
    Choose the cell with the lowest priority  $c_s \in S$ .
    Choose the mass  $M_s$  in  $c_s$  with the lowest priority.
    Set  $q$  to the operation  $choose(M)$ .
    Set  $M_c$  to generate  $(M_s, q)$ .
    if  $M_c = 0$  then
         $priority(M_s) := 2(priority(M_s) + \text{iteration number})$ .
    else
        if  $M_c \cap A \neq 0$  then
            Take  $q_f \in M_c$ .
            Return  $path(M_c, q_f)$ .
        end if
         $priority(M_s) := 2(priority(M_s) + 1)$ .
        while  $(M_c \neq 0)$  do
             $C_{next} := stab(S, M_c)$ .
             $M_r := C_{next} \cap M_c$  and  $M_c := M_c \cap M_r$ .
            if not first time through and  $density(C_{next}) > avgdensity(S)$  then
                Break out of the while loop.
            else
                 $\mathbf{M} := \mathbf{M} \cup \{M_r\}$ .
                 $priority(M_s) := \text{iteration}$ .
            end if
        end while
        Subdivide cell  $c_s$ .
    end if
end for

```

Comments on the pseudo-code

The inputs of this algorithm are the set of goals A , the root state of the tree q_0 , and the number of iterations N . The $priority(C)$ is the priority of each cell and defines according to Equation 2.13.

$$priority(C) = \frac{\min\{priority(M) : M \subset C \text{ and } M \in \mathbf{M}\}}{C} \quad (2.13)$$

The $stab$ function returns a cell that holds the first state of M_c . The density function determines the density of a cell by the sum of the weight of the masses in that cell divided by the volume of the cell. Finally, the $avgdensity$ determine the average density of a cell.

Chapter 3

Methods

This chapter describes the methods used to realize the motion planning algorithms and the benchmark methods used to evaluate the result of motion planning algorithms. This section 3.1 explains the utilized methods and also various modules involved in the motion planning process. Subsection 3.1.1 describes the designed method for the motion planning process in the ROS Ecosystem. Subsection 3.1.2 highlights the underlying structure of the ROS node and the communications methods between them. Section 3.2 demonstrates the method that has been designed to generate a kinematics chain and visualizes and defines object and collision boundary.

Finally, Section 3.3 describes the experiments designed to evaluate and compare the results of the motion planning..

3.1 Motion planning and visualization

Design of various control methods and motion planning algorithms have been implemented in the ROS environment. Hence two workspaces have been created. The first one is the ROS-Industrial Universal Robot meta-package which has been cloned from the ros-industrial GitHub. This meta-package contains the UR5 configuration and other tools required to simulate and create a kinematic description of the UR5 robot.

The second workspace is created to implement the motion planning methods, visualization, and data extraction. This workspace contains four packages and several nodes with different functionalities and services. These nodes communicate with each other using ROS communication tools such as messages, services and action client services (look at Appendix A for more information about ROS communication tools). In the following, these packages and nodes will be explained in detail.

3.1.1 Description of ROS packages and their design purposes

Publisher package

The publisher package's primary function is to read the sensor information concerning the screw holes and publish them as a custom message into a pose marker topic. A node called the publisher.cpp does the reading and publishing task. In addition, the message for transferring the sensor's information has also been declared in this package. This message has a custom type, with a format which is shown in Listing G.1.

Workcell_core package

The Workcell_core package consists of 2 nodes, workcell_node, position_node, and a service declaration that retrieves the robot's position. This service holds a request and the response to this request. This service is called by sending a request of type string, including the name of the base frame and, in response, the service provides the robot's pose. In addition, this package is designed to perform other tasks and functionalities such as:

- Receive the screw hole coordinate by subscribing to the pose marker topic.
- Perform the motion planning process by creating a planning group called manipulator, which is a set of connected links and joints by using Move_group interface (Read more about Move_group interface in Appendix D). This node uses MoveIt capabilities in combination with OMPL features to initialize and perform the robotic manipulation.
- Process, calculate, and publish the result such as planning time, execution time, and planning accuracy in two txt files.

Furthermore, this package holds another node, position_node, that transforms the screw holes coordinates from the camera frame into the robot frame using the tf function (This node is explained in more detail in Section 3.1.2).

Workcell_moveit_config package

All the configurations and specifications related to MoveIt, OMPL, and the robot kinematics, have been declared in this package. For example, motion planners and their configuration, such as range and goal bias, are defined here in the Ompl_planning.YAML file. Another important file in this package is workcell.srdf. This file contains information about various planning groups

and the name and type of the robot joints. The connection between joints and various initial poses for the robot is also defined in this file (see Listing G.3). The launch files are also defined here. Launch files use to run various applications, libraries, and files only by launching one file. This property eases the task as online motion planning as there is no need for running each motion planning components separately in a terminal.

Workcell_support package

Workcell_support package is a supplementary package that contains all the information about the workspace.

The main file in this package is workcell.xacro, which contains the declaration of objects, obstacles, and other environment parts. The specification of objects, such as the collision boundary, and visual properties, such as color and the object's geometry, are also defined in this file. All the objects are represented with a parent-child relationship. The snippet in Listing G.2 shows an example of the child-parent declaration of the object table and the robot. The position of the table (child object) is defined concerning its parent (table). The values for (x,y,z) in the snippet represent the translational displacement between the table and the robot origin, and (r,p,y) represents the relative difference in roll, pitch, and yaw between these two objects.

3.1.2 Description of nodes and their underlying structure

Each ROS package consists of several files, nodes, and complimentary utilities. This section aims to highlight the underlying structure of the nodes and their communication methods. Figure 3.1 demonstrates the connection between nodes and the topics they use to communicate with each other. Each node acts as a function designed to perform one or multiple tasks if the tasks are related to each other. Some of the most important nodes will be explained in the following. Pose_publisher's purpose is to receive the holes coordinates from the sensors and publish them as a message. coordinates are set manually in the code since the sensor implementation is beyond this project's scope. coordinates are of type geometry_msgs/Pose.msg and composed of position in point format and orientation in quaternion format. These coordinates are in the camera frame although, they have to be transformed into the world frame. The transformation takes place in another node called the position_node.

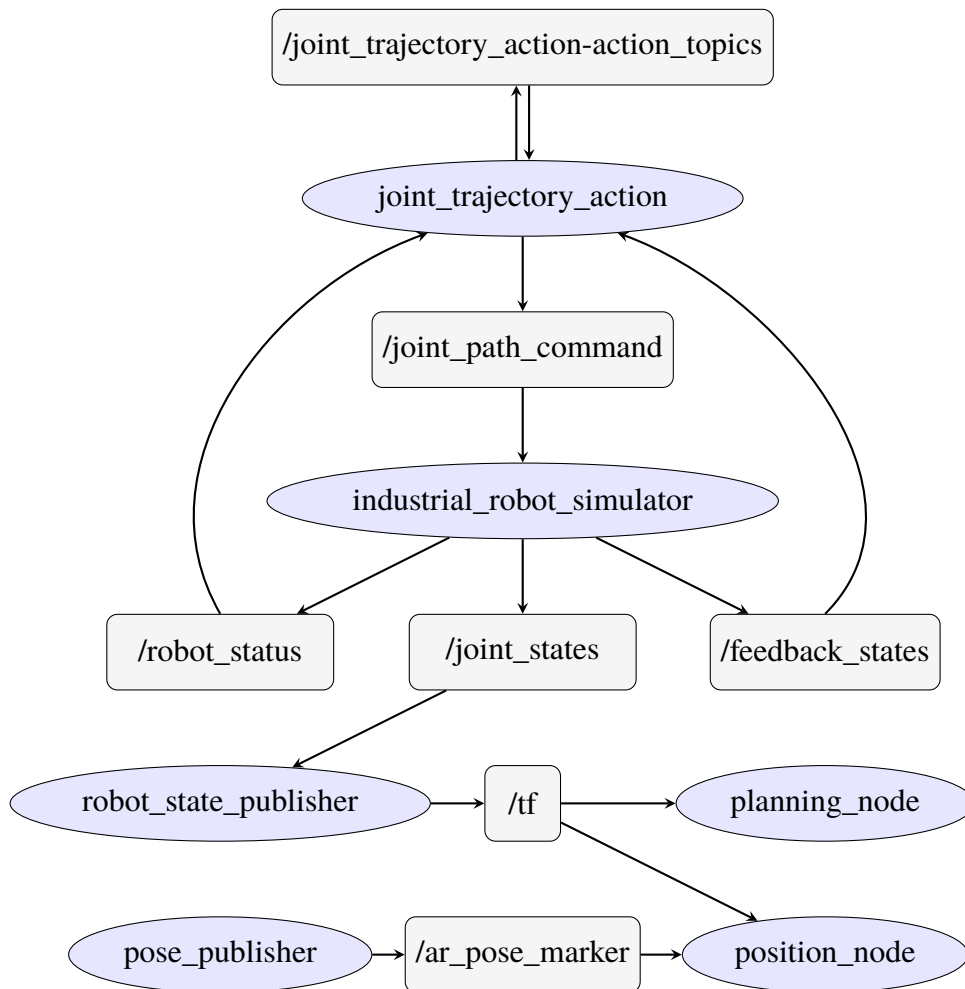


Figure 3.1: Structure of ROS nodes and topics. The ellipses represent the nodes in this figure, and the boxes represent the topics.

Position_node:

The position_node's first task is to subscribe to a topic, pose_marker, and receive the hole coordinates published by the Pose_publisher. When the data is received, they are transformed into the desired frame, the world frame in this case. Tf package is a tool that performs this transformation. Tf package enables users to maintain a relationship between various frames and transform the different data types, such as points and vectors, between any two coordinate frames. Here a service called Localization.srv is declared. This service enables the position_node to exchange data with the planning_node. The planning_node sends a type string request containing the robot frame and receives the transformed coordination in that frame.

Planning_node:

This package has several fundamental tasks and is considered the main module containing all the path planning algorithms. All these tasks will be explained separately in the following:

- **Initialization of the path planning parameters:**

Move_group_interface is used to initialize the planning object, choose the path planner and planner group. In addition, Move_group enables users to define a wide range of tuning parameters such as maximum planning time, planning resolution, etc.

- **Planning and execution of path:**

Move_group provides various Move_group provides various functions to move the end-effector between waypoints. For example, Move_group holds a function called move(), which use to perform planning and executing a path from start to endpoints. These two processes can also call separately using the Plan() and execution() functions. The advantage of separate usage is that the time for planning and execution can be obtained independently, which is helpful to evaluate the performance of motion planners.

- **Data collection and creating data files:**

This node calculates and stores motion planning's row output, including the planning time, execution time, and planning accuracy.

3.2 Defining kinematic, visual and collision boundary of the UR5

Simulating a robot consists of a few steps, including generating a kinematic chain, defining the robot's geometry, visual and collision boundary.

3.2.1 Kinematics chain generation

Configuration files for kinematics chains have been generated using the MoveIt Setup Assistant that receives the [URDF](#) model of a robot and generates a [Semantic Robot Description Format \(SRDF\)](#) file in addition to all the necessary configuration files needed for the MoveIt pipeline. For UR5, a planning group consisting of all six joints has been defined. The process of creating the planning group, adding joints and generating the Self-Collision matrix will be explained in the following:

- **Generate Self-Collision Matrix:**

In this step, the Self-Collision Matrix Generator creates a self-collision matrix by searching for pairs of links and removing some from the collision checking algorithm. The sampling density is the parameter that specifies the number of random positions on the robot to check for self-collision and can be adjusted. A higher value for this parameter enforces more computation time, while a lower value is more likely to remove pairs of links with a high probability of collision.

- **Add Virtual Joints:**

Virtual joints use to attach the robot to the workspace. For example, robot arms often need one virtual joint, representing the motion of the robot's base frame in a world plane.

- **Add Planning Groups:**

Planning groups are different sets of links and joints connected together. Planning groups describe different parts of a robot, including an end-effector and set of joints and links that form a robot arm. In this step, a number of planning parameters can be pre-defined for each planning group. Parameters such as the type of kinematic solver, kinematics solver search resolution, kinematics solver timeout, and kinematics solver attempts are examples of parameters that can be pre-defined in MoveIt Setup Assistant.

3.2.2 Define Object's geometry, visual and collision boundary

The geometry of the robot and objects which are used for visualization and collision checking is described in a URDF file. URDF is an XML format that describes robots and the environmental objects. Two separate meshes have been created for visualization and collision checking.

The visualization mesh is more detailed than the collision checking meshes. A simple URDF code is presented in Listing G.4, consisting of a pair of parent and child links and a joint connecting them. Other features such as material name and color have been also added into this code.

Despite all the utilities the URDF format provides, creating large-scale URDF files containing several components becomes repetitive and time-consuming. Lack of programming capabilities makes URDF files unnecessarily large with several duplicate elements. Elements such as links and joints must duplicate with the same property but slightly different names. This issue was solved by adding the XACRO macro to the URDF file. XACRO macro brings programming concepts such as variables, functions, and classes. XACRO

(XML Macros) is an XML macro language. XACRO constructs shorter and more readable XML.

3.3 Algorithm benchmarking

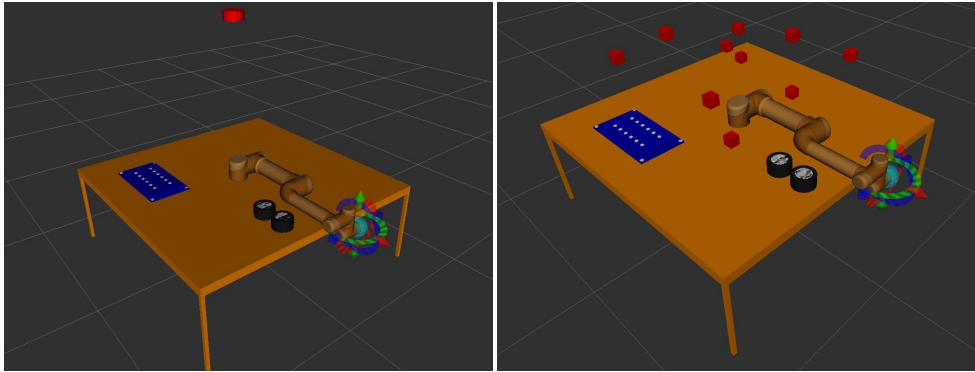
Three benchmark scenarios have been designed to evaluate the behavior of various planners in different conditions. The main distinctive component between the test environments is the number of obstacles. Besides that, all the environmental elements are the same.

Figure 3.2 illustrates all test scenes and the representation of obstacles. The red cube-shaped objects are the obstacles. Scene 1 has no obstacles, scene 2 has ten obstacles, and scene 3 has twenty. Lastly, the blue object on the board represents the radio filter with several white dots indicating the hole positions and, the black cylindrical elements render the screw feeder. The scene in Figure 3.2(b) has been selected for the primary experiment, and for the secondary benchmarking, the motion planners have been studied in all three scenes.

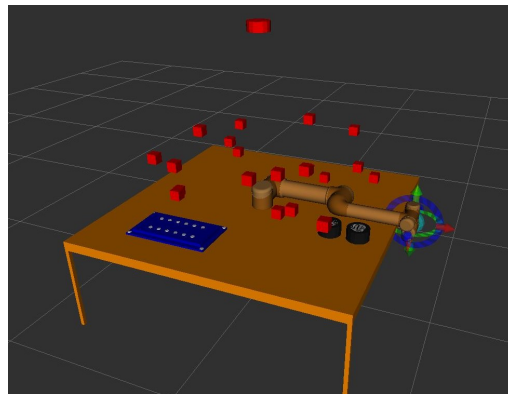
3.3.1 Primary benchmarking

This experiment investigates motion planners' accuracy and time efficiency while performing a screwing task on a radio filter. The screw holes are located at two different heights on the radio filter. The screw holes on the outer edges are on a lower height level and have a wider radius than the middle ones. Hence two screw feeders have also been simulated for each screw type. Figure 3.3 shows that holes at the outer edges require a screw of type 1, and the middle ones require a type 2 screw. Different screw types do not represent any actual screw model. Instead, the purpose of this naming is to demonstrate that there are two distinct screw types for different holes. Moreover, the reason for having two types of screw types in two different heights is to demonstrate and highlight the motion planner's ability to perform 3D tasks.

The motion planning module receives coordinates of the outer holes and the respective screw feeder. Next, the end-effector moves to the screw feeder while avoiding all the obstacles in its surrounding. Eventually, the end-effector moves on to the outer coordinates and this process repeats until all the outer holes are covered. After that, the end-effector receives the coordinates of middle holes and their respective screw feeder and replicates the same process until all the screw holes are visited. The planning times for each pair of consecutive states are collected in an online process and is visible both in a terminal and also in a text file shown in Listing G.5. Moreover, additional



(a) Scene 1, which contains no obstacles (b) Scene 2, medium obstacles density



(c) Scene 3, high obstacles density

Figure 3.2: Three benchmark scenarios with different number of obstacles designed to evaluate the performance of motion planners

results such as total planning time, average plannings and execution time, and average execution time add to the text files when the plannings task is completed. The planning accuracy will also be stored in another file, shown in Listings G.6.

The planning accuracy is the metric difference between the coordination sent to the `move()` function and the actual position of the end-effector. A function named `Difference` is responsible for the computation of the planning accuracy, according to the snippet in Listing G.7.

3.3.2 Second experiment

The idea of the second benchmarking is to decide which motion planner provides the best performance concerning the requirements and goals of this project. The performance of planners is compared to each other, especially

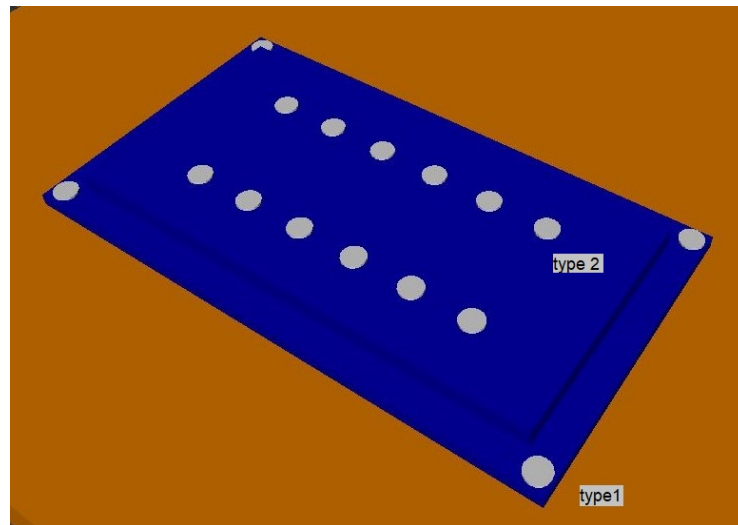


Figure 3.3: A filter unit and with two different screw types for inner holes and outer holes

with the state-of-the-art motion planner RRT-Connect, to check the validity of the research question.

The second experiment can be divided into two parts. The first part is almost similar to the first experiment except that motions planners are tested in all three scenes. The second part extends the primary experiment and provides in-depth analysis and comparative information about the performance of the motion planner. This part is explained in the following:

1. Run a motion planner and connect it to a database.
2. Save a scene as a .scene format file.
3. Define a set of random queries in each scene. A query is characterized as a pair of start and goal states.
4. Create and modify a .yaml configuration file, which contains information about the scenes and queries. The configuration file usually consists of three parts specified by headers. In the first part, the problem is defined with some required parameters such as name, robot, start, and goal states. The second part holds the parameters related to benchmarking process, such as time limit and memory limit. Finally, in the last part the motion planners is declared (See Listing G.8 which demonstrates a simplest version of a configuration file for a RRT planner).

5. Save and convert the output as a .db file. Import the .db file to plannerarena.org provides excellent tools for analyzing and benchmarking the planner's performance.

3.4 Data Collection

There are two output references for each motion planner. The first one, Rviz, is a graphical user interface that visualizes the motion planner's performance in real-time. The main components of a motion planning process are the robot, obstacles, and the radio unit. Rviz visualizes these items with graphical objects with a specific shape and collision boundary defined in a [URDF](#) file. Every time a motion is generated, the motion alongside the environment is stored in a warehouse. These pieces of information are used to evaluate the motion planner's performance by injecting them into more advanced algorithms and analyzing them with more details. This part is explained in more detail in [Section 3.3.2](#).

The second source of data is generated by algorithms defined in the `workcell_node.cpp` file. The output of these algorithms is logged in two pdf files, demonstrating row data for various benchmark attributes such as planning time, execution time, and planning accuracy.

Chapter 4

Results

This chapter presents the result of the motion planners. Section 4.1 presents the result of the primary experiment and Section 4.2 demonstrates the result obtained from the second experiment. For the second experiment, the result is sorted based on the investigated metrics.

4.1 Result of primary experiment

The first experiment investigates the motion planners' accuracy and time efficiency while performing the screwing task designed to be similar to an actual screwing process on a radio filter. Note the obstacles have been added to the environment to test the ability of the motion planners to avoid static objects. In this experiment setup, the end-effector visits the total number of 33 coordinates in each run, corresponding to the number of transitions from the screw feeder to the position of the screw holes and vice versa. This experiment has been repeated ten times for every motion planner with the same queries for the start and goals state in the experimental setup, demonstrated in Figure 3.2(b). The average plannings time and the average metric accuracy are presented in table 4.1 and 4.2.

Planner	Average plan time (s)	Average execution time (s)
EST	0.945608	10.0458
KPIECE	1.61858	10.39328
RRT-Connect	0.892533	8.80015
PDST	1.30422	9.03131

Table 4.1: Average planning time and execution time obtained by the planners in the primary experiment

According to the result presented in Table 4.1, the slowest planning time belongs to the KPIECE with 1.61 seconds, while the RRT-Connect comes on top with almost half of this value. Note that, two planners, RRT-Connect and EST, achieved a planning time of less than one second. Moreover, the state-of-the-art planner RRT-Connect has the best performance compared to the other ones.

Additionally, a correlation between planning and execution time is also noticeable. This correlation is expected since a lengthy planning time often leads to a long path, and extensive paths usually take more time to execute. The only exception that does not follow this assumption is the comparison between PDST and EST planners. PDST has a higher a planning time average than EST's, while its average execution time is lower. The reason for this specific case is that PDST generates a shorter path with a higher level of smoothness. This topic is discussed in section 6.2.

Planner	Average metric accuracy (m)
EST	7.66122e-05
KPIECE	7.64564e-05
RRT-Connect	7.79454e-05
PDST	7.49578e-05

Table 4.2: The metric accuracy obtained by the planners in the primary experiment

The planner's accuracy is shown in Table 4.2 and presents almost the same value for all the planners. This outcome is caused by some hyper-parameters used for creating kinematic chains. One of these parameters with the highest impact on planning accuracy is discretization resolution. Increasing this parameter improves accuracy but makes the motion planner slower. Increasing the resolution of the discretization will increase the density of states in the state space, leading to an overall slower planning time.

4.2 Result of second experiment

The second experiment is an extension of the primary experiment. In this experiment, the motion planners will be investigated in terms of additional metrics such as path length, solution smoothness and memory usage and have been studied in two more environments. The first added environment is demonstrated in 3.2(a) with no obstacles, and the other added scene is shown in Figure 3.2(c) with double amount of the obstacles compared with the original scene used in the first experience. The difference between environments is the number of obstacles which makes the planning problem generally harder to solve as the number of obstacles increases. The result will be presented separately for each planning metric except for the memory usage. The memory usage of different planners and benchmarking scenarios is shown in a single figure.

4.2.1 Planning time

Plan time (s) is defined as the time taken by planners to finish the motion planning process. Figures 4.1 and 4.2 presenting the results for all three test scenarios.

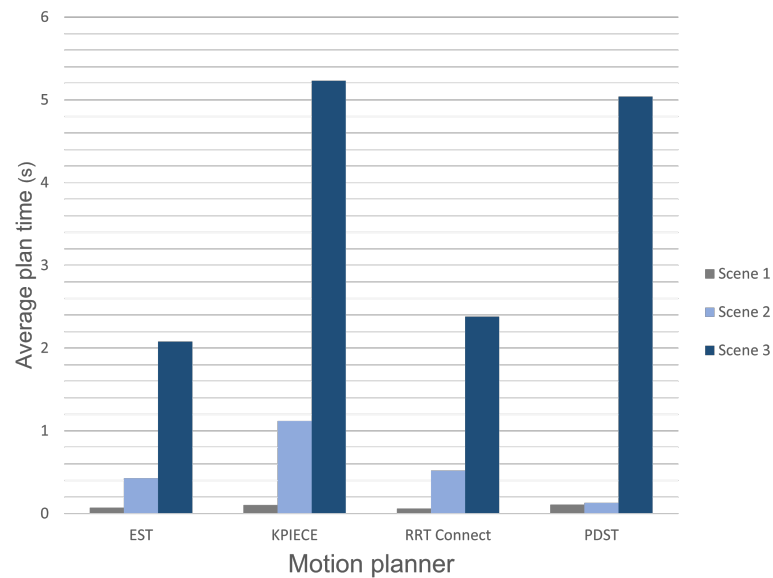


Figure 4.1: Average plan time achieved by the planners.

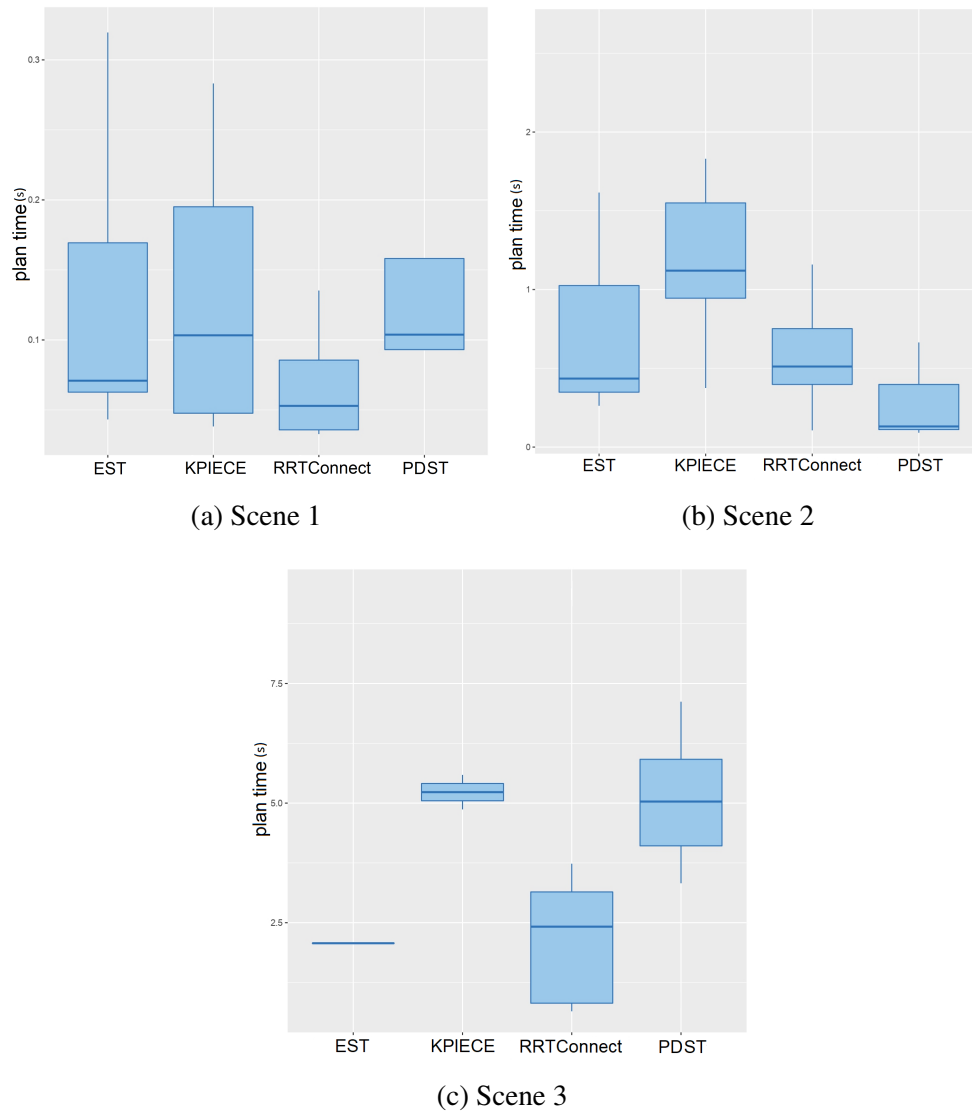


Figure 4.2: Plan time obtained by the planners in the secondary experiment.

The result of the first benchmarking scene is shown in Figure 4.2(a). RRT-Connect is the fastest planner, EST is the second best, KPIECE stands in third place, and PDST is the slowest planner. The solid performances of EST and RRT-Connect can be related to their bidirectional structure, which helps them cover the free space faster, especially if no obstacle is present. Furthermore, RRT is a goal-oriented planner that balances the greedy search and the exploration rate. Hence it has a fast performance in open areas. A few obstacles have been added to the workspace in the second benchmarking scene, and a slightly different result has been obtained. According to the result

presented in Figure 4.2(b), PDST is the fastest planner right above the EST, while RRT-Connect and KPIECE are placed in third and last place. The fast performance of PDST is due to its improved exploration strategy and low-cost forward propagation. In addition, this planner explores the environment by adopting segments instead of samples suitable for cases with few obstacles. Finally, in the final benchmarking scene, only two planners have a ratio of solved cases above 80 percent, while EST has a success ratio of 10 percent, and KPIECE with 20 percent of total runs. For this reason, the comparison will be between RRT-Connect and PDST. In this comparison, PDST is slower than RRT-Connect, according to the result of Figure 4.2(c). PDST optimal performance in a low-density environment could worsen in a more dense environment containing narrow passages. In other words, sampling the narrow passages is faster than adapting segments in areas with a high density of obstacles.

4.2.2 Path length

Path length defines as the summation of the traveled distance for each joint, which derives from $\sum_{k=1}^n |x_k - x_{k0}|$. In this equation n is number of joints and the goal and start positions for each joint are donated by x_k and x_{k0} . Figures 4.3 and 4.4 presents the path length for all three test scenarios.

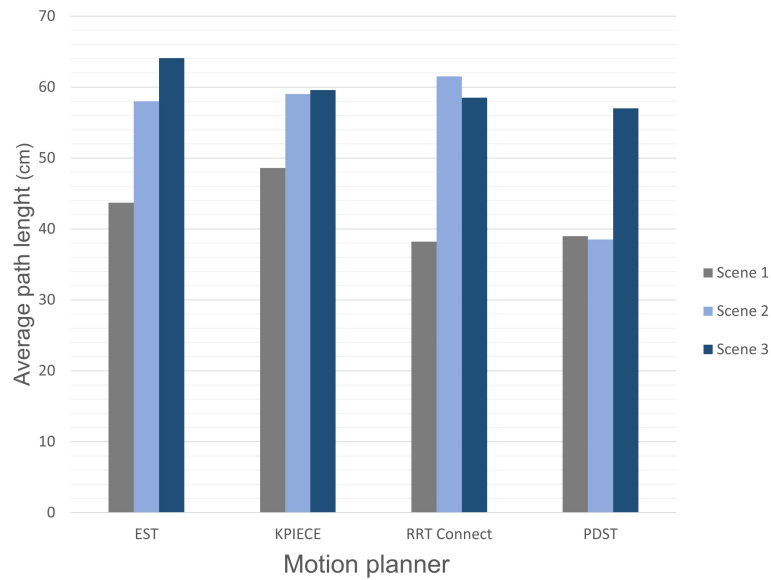


Figure 4.3: Average path length achieved by the planners.

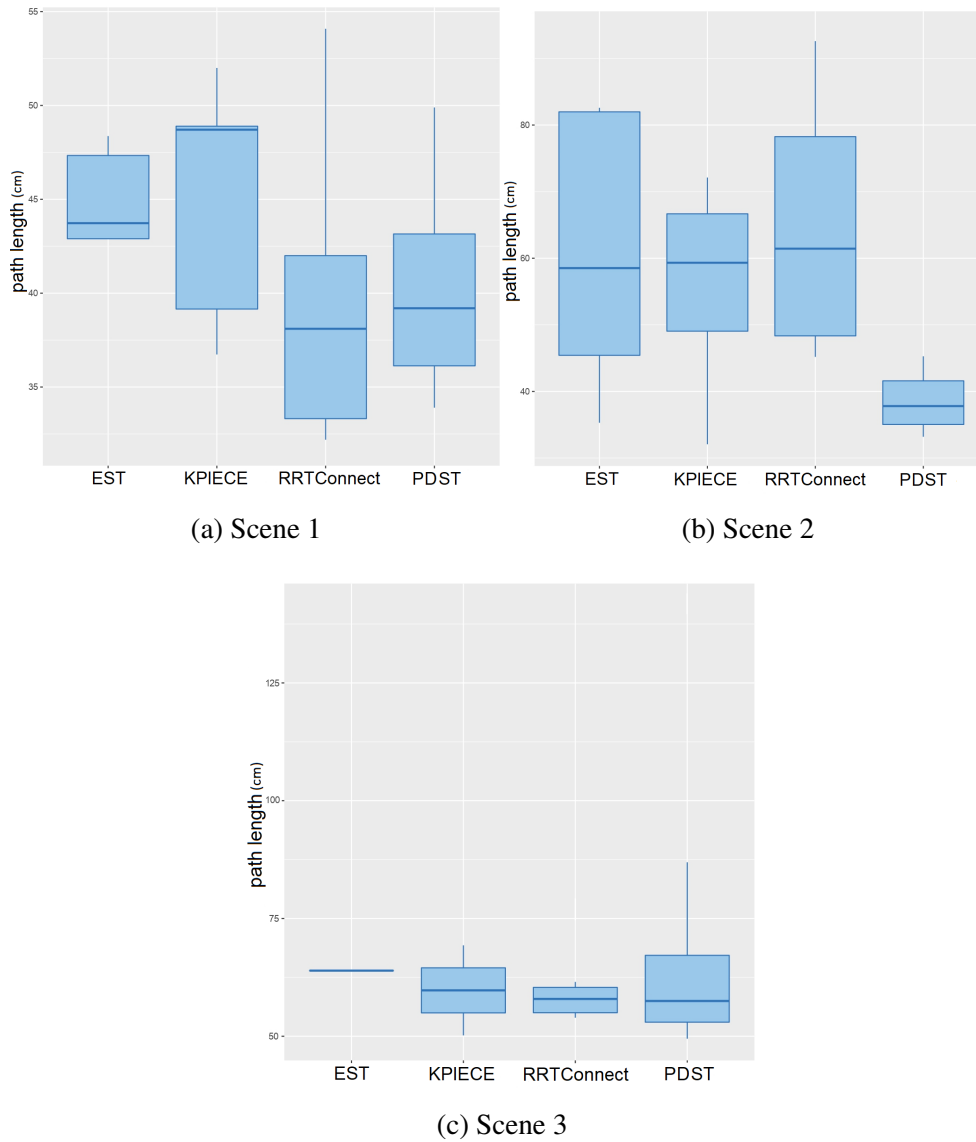


Figure 4.4: Length of the path obtained by the planners in the secondary experiment.

The result of the first benchmarking scenario is shown in Figure 4.4(b). RRT-Connect has the shortest path, and PDST is the second shortest, while EST and KPIECE stand in third and last places.

In the second experiment, obstacles have been added to the workspace, and the result is displayed in Figure 4.4(b). PDST provides the shortest path while RRT-Connect is the planner with the largest path length. Note that the average path length of RRT-Connect, EST, and KPIECE is almost identical, while PDST has created a path with nearly 40 percent less length. One possible

explanation for the PDST excellent performance is its greedy deterministic exploration strategy and using path segments instead of the traditional states. So whenever the workspace does not include any obstacle, PDST creates some straight path segments that connect the start state to the goal state.

Finally, in the final benchmarking scenario, the comparison will be between RRT-Connect and PDST for the same reason mentioned earlier. The average performances of these two planners in terms of path length very close to each other. However, RRT-Connect has less variance in path length while PDST has slightly less average value.

RRT-Connect's stable performance in test scenarios 2 and 3 was not expected. RRT-Connect creates a cubic structured path since nodes are directly connected to their nearest neighbor without further action to smooth these attachments. Moreover, RRT does not usually take the hypotenuse between two-state and instead takes a path of two segments. However, the possible explanation could be that RRT-Connect is suitable for cases with numerous environmental obstacles such that the cubic structure finds a reasonably short path.

4.2.3 Solution smoothness

Solution smoothness indicates the level of smoothness of the created trajectory. Solution smoothness derives from the following formula. The smoothness of a straight line is 0. Hence if the value is closer to 0, the path found is smoother.

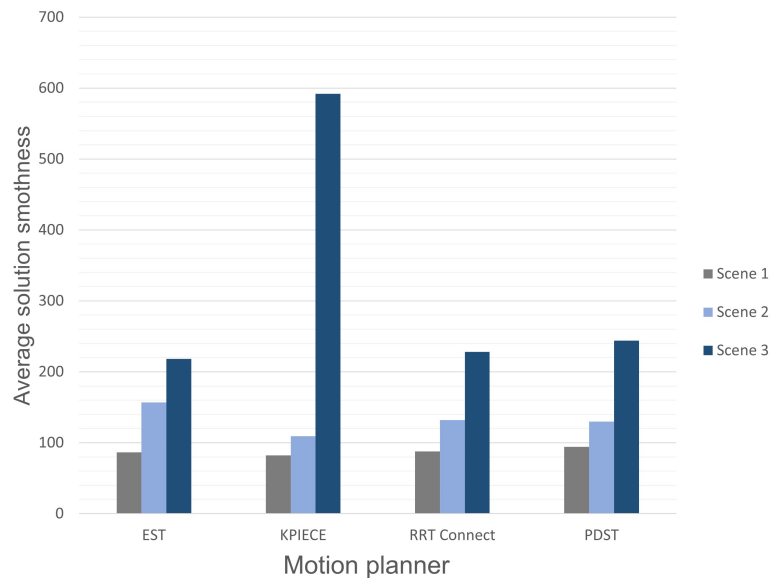


Figure 4.5: Average Solution smoothness achieved up by the planners.

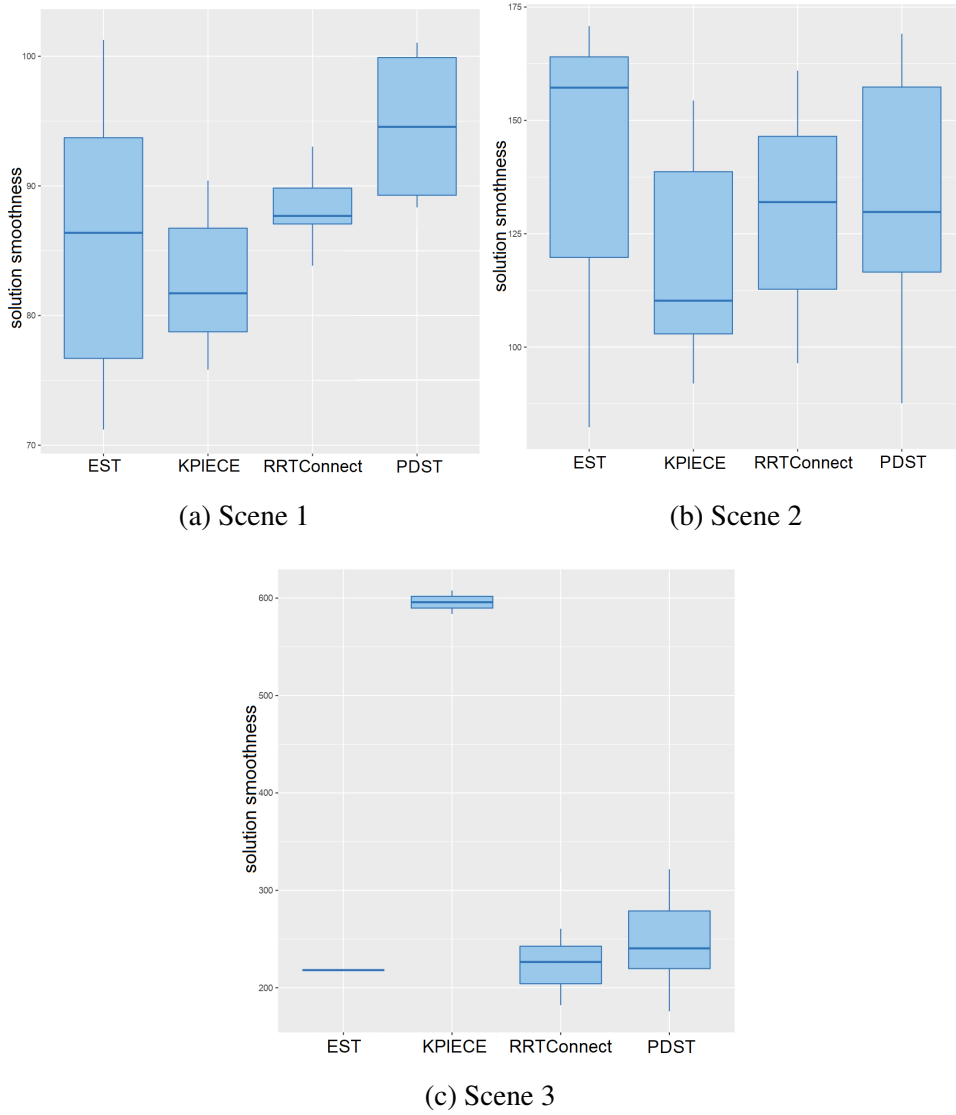


Figure 4.6: The smoothness of the solutions obtained by the planners in the secondary experiment.

$$\sum_{k=2}^{n-1} (2(\pi - \arccos(\frac{(d_{k-2,k-1}^2 + d_{k-1,k}^2 - d_{k-2,k}^2)}{d_{k-2,k-1}d_{k-1,k}})))^2 \quad (4.1)$$

Solution smoothness is a comparative metric and depends on the number of waypoints and their relative position. This value derives by measuring the angle between the path segments of the triangles formed by consecutive path segments and using Pythagora's theorem. The input of this formula is the angle formed between three consecutive waypoints. In this formula, n is the number

of points alongside a trajectory and $d_{x,y}$ donates the distance between point x and point y . The result for solution smoothness is shown in Figures 4.5 and 4.6.

The result of the first benchmarking scenario is shown in Figure 4.6(a). EST provides the highest level of smoothness, while KPIECE is the planner with the lowest level of smoothness. According to the result of the second experiment presented in Figure 4.6(b), EST grants the highest smoothness right above the RRT-Connect, while PDST and KPIECE placed in third and last place. Finally, only two planners solved most of the cases in the final benchmarking scenario, and the comparison will be between RRT-Connect and PDST. Nevertheless, PDST still provides the best performance with a slightly better smoothness level than RRT-Connect. PSTD, in general, provides better path smoothness than its competitors since this method takes advantage of path segments instead of samples. In other words, paths generated by PDST are a union of path segments. Each segment is a straight line that leads to a smooth path with less jerky behavior.

4.2.4 Memory usage

Memory usage indicates the average consumption of the processor's memory in MB during the plannings process. Memory usage is measured by monitoring the CPU usage used for the motion planning process. In addition, the OMPL command-line program provides users with the utilities to limit and calculate memory usage. Memory usage is an essential factor in evaluating the performance of a motion planner since memory usage indicates how computational heavy is every motion planner. Heavy computation's task requires the CPU to work with a higher frequency which in turn influence the energy consumption of the system. The values for memory usage of motion planners in different test scenarios are presented in Figure 4.7.

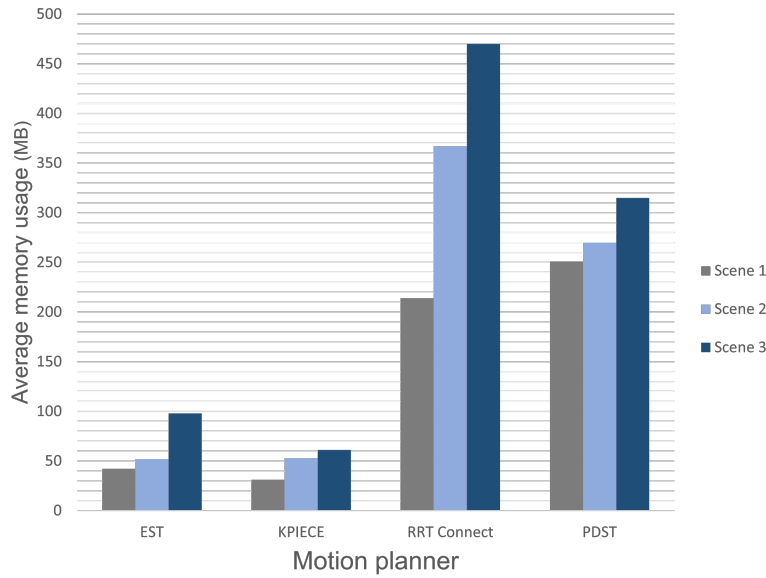


Figure 4.7: Average memory taken up by each planner.

The average memory taken up by the planners in the second experiment is shown in Figure 4.7. EST and KPIECE are the planners with the least amount of memory usage. KPIECE takes up the least memory, and EST is the second memory-efficient planner. KPIECE's main advantage is the computational improvement compared to other planners by utilizing a shared-memory implementation and the ability of parallel computation, as mentioned in Section 2.6.3. Similarly, EST uses memory capacity efficiently by taking advantage of the simple expansion and connection algorithm structure. On the other hand, RRT and PDST underperformed in terms of memory usage. One possible explanation for RRT-Connect's non-optimal performance is that RRT-Connect takes so much memory capacity to update the tree and keeps track of all the nodes and edges to rewire the connection between nodes. PDST suffers from memory inefficiency as well for the dynamical system. An increase in the number of iterations will also increase the number of states needed to build the tree and exceed the core size and memory usage [23].

4.2.5 Number of solved cases

The number of solved cases is determined by the number of runs a motion planner finds a solution for all the queries. The results are presented in Tables 4.3, 4.4 and 4.5. The planning threshold set to 10 seconds for all the planners

in all the scenes. Thus if a planner exceeds this threshold, the planner fails, and the planning output will be a fail attempt.

Planner	Missing	Total runs
EST	0	10
KPIECE	0	10
RRT-Connect	0	10
PDST	0	10

Table 4.3: Number of solved cases for each planner in the first test scenario of the second experiment.

Planner	Missing	Total runs
EST	0	10
KPIECE	1	10
RRT-Connect	0	10
PDST	0	10

Table 4.4: Number of solved cases for each planner in the second test scenario of the second experiment.

Planner	Missing	Total runs
EST	9	10
KPIECE	8	10
RRT-Connect	0	10
PDST	0	10

Table 4.5: Number of solved cases for each planner in the third test scenario of the second experiment.

According to Tables 4.3, 4.4, and 4.5, all the planners solved the planning problem in the first benchmarking scene. As a result, the planners successfully find a path for all the queries within the maximum planning threshold. As obstacles have been added to the environment in the second scene, KPIECE fails one of the ten attempts while the other planner passes all the runs. As the number of obstacles increases in the third scene, more planners start to fail. For example, EST fails nine attempts and KPIECE nine times. However, RRT-Connect and PDST have managed to pass all ten attempts and are considered

the optimal planner to handle obstacles in terms of successful attempts. In conclusion, for an environment with a high density of obstacles, RRT-Connect and PDST are superior options. However, in the other cases where the obstacles do not exist or have a low density, the choice of a motion planner is due to other performance metrics since all the planners passed the test cases without a single fail.

4.2.6 Summary of the motion planners performance

Figure 4.8 summarizes the relative performance of different motion planners in the second experience. The purpose of these radar charts is to visualize the relative performance of every planner in each test scenario, and the values in this chart do not accurately represent the actual performance. In this representation, a higher value for each metric is desirable. Hence the outer boundary of each chart is the ideal performance line.

Note that in Sub-figure 4.8c, KPIECE and EST are excluded since they fail more than 80 percent of the motion planning runs in this test scenario.

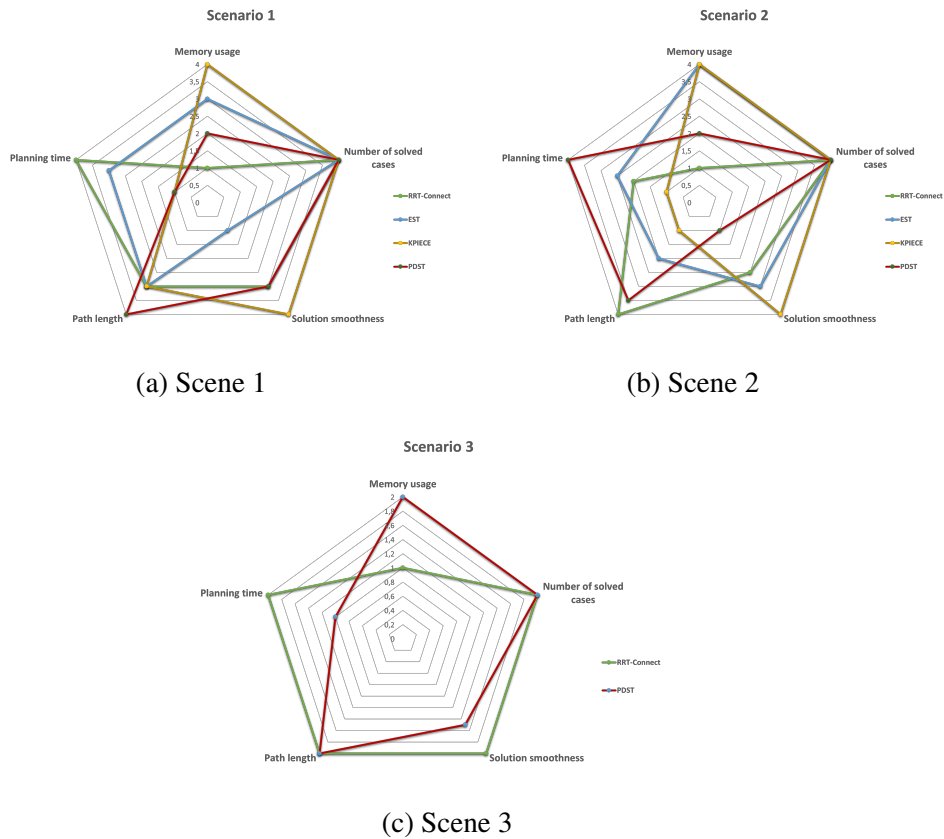


Figure 4.8: The relative performance of the motion planners.

Chapter 5

Discussion

In this chapter, section 5.1 explains the implications of the results, and section 5.3 will discuss the Other possible methods to solve the screwing process. Finally section 5.3 will cover the reliability of the collected data.

5.1 Implications of the results

One of the main objectives of this project is to find the planners can outperform the state of the are motion planner RRT-Connect especially in terms of time. The planning time is undeniably the most important metric, especially for the robots that aim to work in real-time. On the other hand, the motion planners fail for mainly two reasons: the inability to find a collision-free path between two waypoints or pass the time limit assigned for the planning time. In our cases, almost all the fail attempts occurred because of the time limit since different motion planning algorithms stop searching for a feasible path and return failure as soon as the time limit is reached. So the planning time also impacts the overall performance. Hence, we can state not all planners provide an optimal performance regarding the planning time. For the cases that time is an important metric, KPIECE and EST are not grantees to provide acceptable performance.

on the other hand, two planners, RRT-Connect and PDST, have passed all the experiments with a 100 percent success rate. This result is significant since this experiment tries to mimic an actual screwing process which provides a fast and reliable performance to plan and execute real-time motion planning. Consequently, these two planners can be implemented on an actual robot arm and perform the screwing process in various environments with minor adjustments concerning real-world disturbances and the robot limitation. Another insight from the obtained results is that all the planners can plan a

motion in three-dimensional space. The planning process is not limited to a two-dimensional space as the holes are placed at different distances from the XY plane.

5.2 Another conceivable method

Some planning problems can be solved by another more practical method by reducing the number of dimensions in the planning problem. The workspace of the majority of planning problems is a three-dimensional space. However, one possible simplification is to assume the height to be fixed, streamline the motion planning process to a two-dimensional problem, and add the vertical transition to change the high level in points of interest.

In other words, the holes' coordinates project on a plane parallel with the radio surface. The planner plans a motion between the projection of the waypoints in this plane and then moves the end-effector vertically up or down to the actual height at the projection points to reach the right height. This simplification affects the method evaluations as it reduces the complexity of the problem.

This method has its strengths and weaknesses. In this method, there is more focus on the performances in the 2D space. The planning problem is solved efficiently with less complexity in a 2D workspace and expands to the 3D workspace by moving the end effector up and down in the Z direction in both start and endpoints. The weakness of this method is that it can not cover the planning problems that the vertical motion is not possible everywhere in the workspace because of either the robot's physical limitation or the placement of environmental objects and obstacles.

5.3 Assessing reliability of the data

The output of motion planners is probabilistic since the planners themselves are sampling-based that adding variance into our result. Therefore the outputs can vary to a certain extent in different runs. However, the influence of the probabilistic characteristics of the collected data has been minimized by repeating the planning process ten times for the same setup, with each run containing several queries of the same coordination.

Concerning the bias in our results, the simulation of the robot model returns an output that has not been affected by any external or internal disturbances and has minimal bias. However, implementing the same motion planning algorithm on a real robot will probably produce a slightly different result due to the factors mentioned in the previous paragraph.

Regarding the memory usage, the measurement of this metric is not highly

unbiased in OMPL. However, the level of accuracy is still sufficient for the sake of comparison between motion planners. Furthermore, suppose a benchmarking process executes several runs of motion planning directly after each other. In that case, the values for memory usage can be incorrect because the memory usage measurement is based on increased memory usage and not its absolute value. If the memory held by the previous run is not freed completely. So the increase in memory usage may be less than the actual value and even close to zero. This issue has been solved by setting the number of runs to one and running the process multiple times.

Chapter 6

Conclusions and Future work

This chapter summarizes the thesis, presents the conclusion, and suggests areas and topics for future work. This section starts with a summary of the thesis. Then, in section 6.2, conclusions are presented, and limitations are discussed in section 6.3. Finally, section 6.4 describes potential improvements and topics for future work.

6.1 Summary

This project covers the implementation and benchmarking of several motion planners to decide which planner provides optimal performance in different environments. Four control-based motion planners have been implemented in the ROS ecosystem using the MoveIt platform. The performance of the planners was evaluated by designing two experiments in three benchmarking scenarios. The intention of the first test was to determine how the planners can perform a task designed similar to an actual screwing process. The purpose of the second experiment was to investigate how the planners will perform as the density of the obstacles increase. The performances of the planners have been analyzed and compared with each other using different benchmarking tools such as the planner arena. In the next section, the conclusions of the results and findings of this project will be presented.

6.2 Conclusions

In general, the performance of motion planners depends on two important factors, the dynamic of the system and the properties of the environment. UR5 is the only robot model in this project, while the environment is changed in various experiments. Several motion planners have intentionally been

designed to perform optimally in some circumstances with the cost of bad performance in other areas. Moreover, some planning algorithms' design philosophy emphasizes obtaining an excellent performance concerning one metric and less emphasis on the others.

Thus, in our case, choosing a motion planner is a matter of environment properties and the metrics of interest. According to the obtained result from the primary and second experiments regarding planning time, RRT-Connect is the planner with the best overall performance. Moreover, in terms of memory usage and solution smoothness, KPIECE provides the best performance, and EST outperforms RRT-Connect in terms of path length in most of the scenarios. Finally, in terms of solved cases, RRT-Connect and PDST are the planners without a single fail during all experiments.

The research question of this project can be evaluated by investigating Figure 4.8 and also result from the primary experiment. We can state that KPIECE can outperform RRT-Connect in terms of memory usage and solution smoothness in an environment with few obstacles. EST also had some advantages in memory usage and path length in an environment with a low density of obstacles. In summary, in environments with non or a few obstacles, KPIECE, EST and PDST can outperform RRT-Connect to some extent. However, RRT-Connect is still the superior option in a more complicated environment.

Furthermore, an autonomous system needs a motion planner with an overall robust and viable performance in almost all the scenarios and concerning different metrics. Based on the result, RRT-Connect and PSTD are viable solutions to the planning problem since they successfully passed all the tests and provided reliable performances in different settings.

6.3 Future work

The results of this project are theoretical and simulation-based. The next step is to test the designed motion planning module on an actual robot and observe how the planners perform in real-world scenarios. Motion planners probably require adjustment of planning parameters to take the environmental disturbances and the actual robots limitation into account to achieve optimal planning process.

Furthermore, this process still lacks an intelligence module to decide the order of screwing, which is necessary for screwing radio units.

Another topic for future research is investigating the performance of other planners, such as optimizing planners, multi-query planners, and geometric planners.

Another potential improvement is to use a non-Opml planner as a post-

processor to the [OMPL](#) planner. For instance, Covariant Hamiltonian optimization for motion planning (CHOMP) or stochastic Trajectory Optimization for Motion Planning (STOMP) are great options for this purpose.

6.4 Reflections

This paper presents several options for optimal and robust motion planning to perform a screwing task for an industrial robot. Although this project aims to be as general as possible, the results could apply to similar tasks requiring a motion planner. Furthermore, the design fundamental of this project reflected itself and influenced the chosen method and the fashion they have been implemented.

References

- [1] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, “Motion planning networks: Bridging the gap between learning-based and classical motion planners,” *CoRR*, vol. abs/1907.06013, 2019. [Online]. Available: <http://arxiv.org/abs/1907.06013>
- [2] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continuum learning,” *CoRR*, vol. abs/1706.08840, 2017. [Online]. Available: <http://arxiv.org/abs/1706.08840>
- [3] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *CoRR*, vol. abs/1105.1186, 2011. [Online]. Available: <http://arxiv.org/abs/1105.1186>
- [4] ———, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011. doi: 10.1177/0278364911406761. [Online]. Available: <https://doi.org/10.1177/0278364911406761>
- [5] L. Kavraki, P. Svestka, J.-C. Latombe, and M. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996. doi: 10.1109/70.508439
- [6] I. A. sucu and L. E. Kavraki, “Kinodynamic motion planning by interior-exterior cell exploration,” in *Algorithmic Foundation Of Robotics VIII*. Springer, 2009, pp. 449–464.
- [7] J. Phillips, N. Bedrossian, and L. Kavraki, “Guided expansive spaces trees: a search strategy for motion- and cost-constrained state spaces,” vol. 2004, 01 2004. doi: 10.1109/ROBOT.2004.1308890. ISBN 0-7803-8232-3 pp. 3968 – 3973 Vol.4.

- [8] J. Angeles, *Rational Kinematics*, ser. Springer Tracts in Natural Philosophy. Springer New York, 2013. ISBN 9781461239161. [Online]. Available: <https://books.google.se/books?id=WUXTBwAAQBAJ>
- [9] V. Kodiyalam, “Rotations and quaternions,” *The Institute of Mathematical Sciences, Chennai, India*.
- [10] Y.-B. Jia, “Quaternions and rotations,” *Com S*, vol. 477, no. 577, 2008.
- [11] M. Rickert, A. Sieverling, and O. Brock, “Balancing exploration and exploitation in sampling-based motion planning,” *IEEE Transactions on Robotics*, vol. 30, pp. 1305–1317, 12 2014. doi: 10.1109/TRO.2014.2340191
- [12] S. M. LaValle, *Planning Algorithms*. USA: Cambridge University Press, 2006. ISBN 0521862051
- [13] M. Cirnu and I. Badralexi, “On newton-raphson method,” *Romanian Economic Business Review*, vol. 5, pp. 91–94, 01 1995.
- [14] G. Carbone and F. Bravo, *Motion and Operation Planning of Robotic Systems: Background and Practical Approaches*, 01 2015, vol. 29. ISBN 978-3-319-14704-8
- [15] A. Koubaa, *Robot Operating System (ROS): The Complete Reference (Volume 1)*, ser. Studies in Computational Intelligence. Springer International Publishing, 2018. ISBN 9783319798844. [Online]. Available: <https://books.google.se/books?id=2WWZtQEACAAJ>
- [16] I. Sucan, M. Moll, and E. Kavraki, “The open motion planning library,” *Robotics & Automation Magazine, IEEE*, vol. 19, 12 2012. doi: 10.1109/MRA.2012.2205651
- [17] J. Kuffner and S. Lavalley, “Rrt-connect: an efficient approach to single-query path planning,” *Proceedings - IEEE International Conference on Robotics and Automation*, vol. 2, pp. 995–1001, Dec. 2000, iCRA 2000: IEEE International Conference on Robotics and Automation ; Conference date: 24-04-2000 Through 28-04-2000.
- [18] S. M. LaValle and J. James J. Kuffner, “Randomized kinodynamic planning,” *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001. doi: 10.1177/02783640122067453. [Online]. Available: <https://doi.org/10.1177/02783640122067453>

- [19] S. M. LaValle, “Rapidly-exploring random trees : a new tool for path planning,” *The annual research report*, 1998.
- [20] D. Hsu, R. Kindel, J.-c. Latombe, and S. Rock, “Randomized kinodynamic motion planning with moving obstacles,” *International Journal of Robotics Research*, vol. 21, 01 2002. doi: 10.1177/027836402320556421
- [21] D. Hsu, J.-C. Latombe, and R. Motwani, “Path planning in expansive configuration spaces,” in *Proceedings of International Conference on Robotics and Automation*, vol. 3, 1997. doi: 10.1109/ROBOT.1997.619371 pp. 2719–2726 vol.3.
- [22] I. A. Sucan and L. E. Kavraki, “A sampling-based tree planner for systems with complex dynamics,” *IEEE Transactions on Robotics*, vol. 28, no. 1, pp. 116–131, 2012. doi: 10.1109/TRO.2011.2160466
- [23] A. M. Ladd and L. E. Kavraki, “Motion planning in the presence of drift, underactuation and discrete system changes,” in *Robotics: Science and Systems*, 2005.
- [24] A. Ladd and L. Kavraki, “Fast tree-based exploration of state space for robots with dynamics,” vol. 17, 10 2005.
- [25] D. Coleman, I. Sucan, S. Chitta, and N. Correll, “Reducing the barrier to entry of complex robotic software: a moveit! case study,” 2014.
- [26] X. Zhang, S. Redon, M. Lee, and Y. Kim, “Continuous collision detection for articulated models using taylor models and temporal culling,” *Proceedings of the ACM SIGGRAPH Conference on Computer Graphics*, 08 2007. doi: 10.1145/1275808.1276396
- [27] J. Pan, S. Chitta, and D. Manocha, “Fcl: A general purpose library for collision and proximity queries,” in *2012 IEEE International Conference on Robotics and Automation*, 2012. doi: 10.1109/ICRA.2012.6225337 pp. 3859–3866.
- [28] K. Khokar, P. Beeson, and R. Burrige, “Implementation of kdl inverse kinematics routine on the atlas humanoid robot,” *Procedia Computer Science*, vol. 46, 12 2015. doi: 10.1016/j.procs.2015.02.063
- [29] H. Kam, S.-H. Lee, T. Park, and C.-H. Kim, “Rviz: a toolkit for real domain data visualization,” *Telecommunication Systems*, vol. 60, pp. 1–9, 10 2015. doi: 10.1007/s11235-015-0034-5

- [30] K. L. R. University, “Open motion planning library: A primer,” 2021.
[Online]. Available: https://ompl.kavrakilab.org/OMPL_Primer.pdf

Appendix A

Essential concepts of ROS

The primary source of confusion for someone who wants to learn and work with **ROS** is understanding how different ROS parts come together. Hence, it is important to understand the underlying mechanism that manages communication and controls various parts.

A ROS network consists of a ROS master and multiple ROS nodes. Each node serves as a function performing a specific task. Nodes communicate with each other by exchanging topics, services, and parameter servers. Robotic projects contain several nodes. For instance, one node reads sensor information and publishes it to a topic, and another node detects obstacles, etc.

ROS master provides services such as registration and naming for all the nodes that are often referred to as Roscore. In addition, ROS Master keeps track of topics, services, publishers, and subscribers. These services enable individual nodes to locate and communicate with one another in a peer-to-peer manner. As mentioned above, ROS provides a large variety of communication tools which are listed in the following. All these methods have certain advantages which make them suitable for different tasks and situations [15].

Message

The main method available for nodes for communication is messages ¹. A message is the simplest data structure containing a various collection of built-in types. Standard primitive types such as integer, float, Boolean, and more advanced data structures such as are arrays of primitive types are also supported. Note that messages can also have an arbitrarily nested form.

¹<http://wiki.ros.org/Master>

Service

A Service is established on a request and response relationship, providing a flexible communication tool. This model requires exchanges of a pair of messages. The client sends a request message and waits for the target resource to respond, and the publisher answers the service by sending a reply message. Services are a many-to-many relationship, which means several nodes can simultaneously publish and subscribe to the same service.

Action

Action is another communication mechanism Added to **ROS** in addition to topics and services for long-running tasks. Action server enables a client to receive continuous feedback and stop the service at any point in time.

Appendix B

Flexible Collision Library

Collision checking in motion planning problems is usually the most expensive and time-consuming part. Hence employing a fast and reliable collision checker will improve the motion planning efficiency significantly. **FCL** uses the allowed collision matrix defined by the user to specify the permitted pairs of bodies. These pairs of bodies do not need to be checked for collision during runtime. This feature will save a significant amount of time in the motion planning process. The Allowed Collision Matrix can also be generated and configured by MoveIt! Setup Assistant [25].

Flexible Collision Library is the default library for collision checking in MoveIt. The plugin architecture of the FCL library allows it to integrate with **OMPL** and MoveIt smoothly. FCL provides state-of-the-art methods for collision checking, including the ability to perform the **Continuous Collision Detection (CCD)** [26], discrete collision detection, continuous collision detection, separation distance computation and penetration depth estimation [27]. FCL also integrates easily with motion planners that are fully templated in the C++ programming language.

FCL structure consists of three main parts shown in Figure B.1. In this hierarchical structure, Collision and proximity queries describe a traversal process with objects as the input and collision result as the output [27]. Moreover, different queries with different data types and planning strategies will share the same framework in this traversal structure. FCL handles both solid and deformable objects. Solid objects, including basic geometric shapes such as boxes, spheres, cylinders, and also unstructured triangle meshes. A triangle mesh in this hierarchical data structure is a method of representing objects and their boundaries. For example, a box is modeled as a single-level hierarchy with a single node in a basic geometric shape. Other forms of geometric objects are modeled using a bounding volume hierarchy.

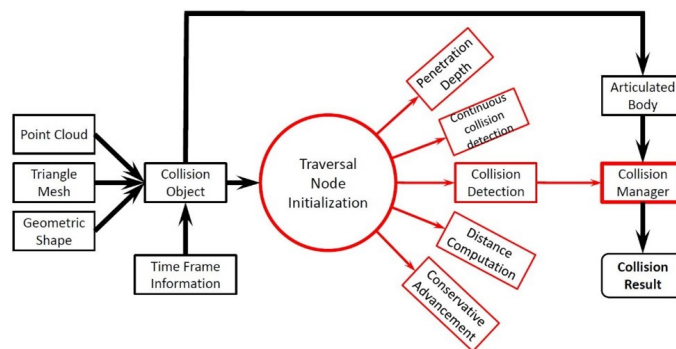


Figure B.1: FCL structure

Appendix C

KDL Inverse Kinematic solver

[KDL](#) has been developed by the Open Robotics Control Software (OROCOS) as the default kinematics plugin for MoveIt to solve inverse kinematics for various kinematic structures. KDL supports only chain kinematics and automatically verifies if the planning group is a chain kinematic or a collection of disjoint joints. KDL solves the linear optimization problem and derives the joint configuration applying the gradient descent minimization [NR](#) method.[\[28\]](#).

Appendix D

Move_group_interface

Move_group_interface is a simple user interface that uses a C++ API and primarily targets advanced users to develop complex functionalities. It provides easy-to-use capabilities for different tasks to develop and debug robotics tasks. These include setting the value to the joint, starting and goal pose, moving the robot arm in the workspace, and adding objects into the environment. The main class in the Move_group_interface is move_group class, which includes several functions performing various tasks. Functions such as move(), execute(), and plan() are used to plan and move the end effector into the desired pose while other functions such as setJointValueTarget(), setEndEffector(), and setPoseTarget enable users to set values into the move_group. There is another group of functions that are used to get robots' information and their pose. For instance, getEndEffector() returns the current end-effector's name and getPoseTargets() returns the pose of the end effector. For instance, moving the end-effector to the desired pose can be done by first defining a target pose in a geometry message. Subsequently, call the setPoseTarget() function to set the goal pose of the end-effector. Next, the plan() function plans a motion from the current state to the specified target as soon as the pose target is set. Finally, the call() function receives a plan and executes it while waiting for completion. This process returns true on success and false on failure. The snippet [D.1](#) demonstrates the initialization of Move_group and all the steps taken by a motion planner to plan and execute a path using the MoveGroup Interface. This code is part of the workcell node from this project, demonstrating all the steps needed to move the end effector. Note that Move_group provides also other powerful features to set the planning time and choose the type of motion planner. [15]

```

1  static const std::string PLANNING_GROUP = "manipulator";
2  moveit::planning_interface::
3  MoveGroupInterface move_group(PLANNING_GROUP);
4  move_group.setPoseReferenceFrame(base_frame);
5  move_group.setPlannerId("RRTConnectkConfigDefault");
6  move_group.setPlanningTime(10);
7  move_group.setPoseTarget(target_pose);
8  moveit::planning_interface::
9  MoveGroupInterface::Plan my_plan;
10 target_pose1.position.x = 0.45;
11 target_pose1.position.y = -0.15;
12 target_pose1.position.z = 0.6;
13 target_pose1.orientation.w = 0;
14 target_pose1.orientation.x = 0;
15 target_pose1.orientation.y = 1;
16 target_pose1.orientation.z = 0;
17 group.setPoseTarget(target_pose1);
18 moveit::planning_interface::MoveGroup::Plan my_plan;
19 bool success = group.plan(my_plan);

```

Listing D.1: Motion planning using Move_group interface

Appendix E

Rviz GUI

Rviz [Graphical User Interface \(GUI\)](#) [29] is the primary tool for visualization in ROS and is suitable for demonstrations and evaluation of motion planning tasks. The Rviz plugin allows users to create environments and define start and goal poses for the robot. Adding the motion planning plugin enables users to plan and execute motion planning tasks with various motion planners and observe the result immediately in MoveIt. Moreover, MoveIt allows visualizing the planning result, saving the planning scene, and importing objects into the workspace.

Appendix F

OMPL

The modular structure of OMPL is a clever design choice as different sampling-based motion planners share similar concepts.

Figure F.1 [16] is showing the connection between different parts of the OMPL library. Some of the most important concepts that are commonly used in almost all the motion planners are discussed in the following subsection in more detail.

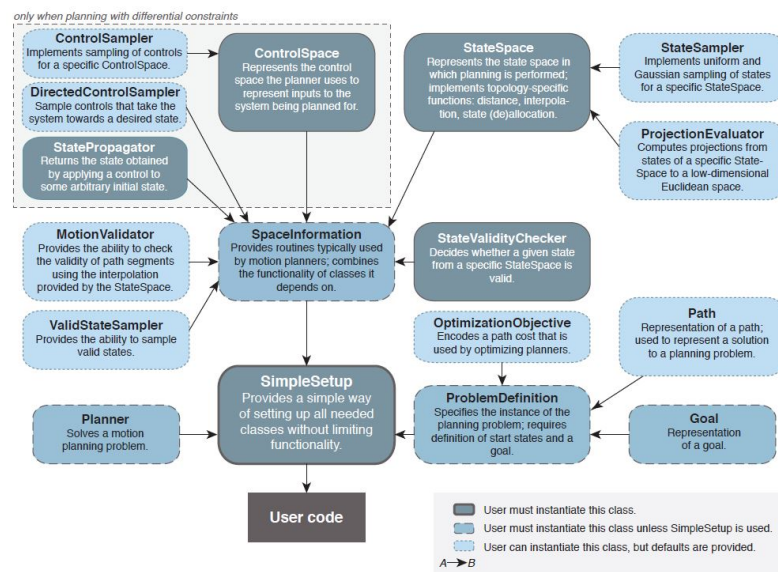


Figure F.1: OMPL structure.

State Sampler

The primary function of The State Sampler is to sample the workspace. State sampler samples the workspace by Gaussian sampling techniques.

There are two types of state samplers, state-space sample and valid state-space samplers. The state-space sampler samples the state space uniformly in the neighborhood of other valid states in the state space. The valid state samplers use the state space samplers solely as a base ground for the sampling process. In addition, the valid state-space sampler generates samples until a valid state is found or the maximum number of samples exceeds a certain threshold. Note that the validity of a state is determined through another library called state Validity Checker.

State Validity Checker

The State Validity Checker's task is to determine the validity of the samples generated by the state sampler. A state is valid if it fulfills two conditions. Firstly, The state must be in obstacles free space, and secondly, the state must be executable for the robot, and the robot's physical limitations and constraints are respected.

OMPL does not include a default collision checker. Hence the user selects a collision checker based on the problem specification and limits. The state validator is an integral part of any motion planning process. Therefore, the developer should use this module as a callback to the motion planner to ensure the generated path's feasibility and avoid any collision with obstacles [30].

State propagator

All the planners investigated in this thesis are control-based. One introductory class for every planner with control inputs is the State propagator. In motion planning with control inputs, the planner needs to access the evaluation of the system for a certain amount of time, starting from a given state as a particular control input is applied to the system.

In addition to checking the validity of two sequential states, it is also required to evaluate how the robot's motion changes in time during the transition from the first state into the second one. In other words, although the start and the end states of a motion are valid, it is also crucial to make sure that the motion remains valid during this transition phase meanwhile the external control input is implied.

Motion Validator

The main responsibility of a motion validator is to check the validity of the motion based on the specified resolution. Conversely, this class determines if the robot's motion during the transition from one valid state to another valid state is collision-free and respects all the velocity limitations of the robot. OMPL uses the Discrete Motion Validator class to interpolate the robot's motion between two valid states in the State Space [30]. Subsequently, it determines if this motion is valid and respects all the constraints.

Local planner

The control-based motion planners use the local planner to compute the local evolution of the robot's motion as a function of time. This motion could be either forward or backward.

In cases when the planner plans only based on the geometric constraints and the local planner act as the motion validator and performs the interpolation between various states in the state space [16].

Appendix G

Listing

```
1  Header header
2  uint32 id
3  geometry_msgs/PoseWithCovariance pose
4  uint32 confidence
```

Listing G.1: Message format uses for transferring the sensor's information

```
1  <joint name="table_to_robot" type="fixed">
2    <parent link="table"/>
3    <child link="base_link"/>
4    <origin xyz="0 0 0" rpy="0.5 0 0"/>
5  </joint>
```

Listing G.2: Example of a child-parents relationship between the table and the robot.

```

1  <dis_col link1="base"    link2="arm"    reason="Adjacent" />
2  <dis_col link1="base"    link2="table"   reason="Adjacent" />
3  <dis_col link1="ee"      link2="wrist1"  reason="never" />
4  <dis_col link1="ee"      link2="wrist2"  reason="never" />
5  <dis_col link1="ee"      link2="wrist3"  reason="Adjacent" />
6  <dis_col link1="forarm"  link2="upper"  reason="Adjacent" />
7  <dis_col link1="forarm"  link2="wrist1"  reason="Adjacent" />
8  <dis_col link1="arm"     link2="table"   reason="Adjacent" />
9  <dis_col link1="arm"     link2="upper"  reason="Adjacent" />
10 <dis_col link1="wrist1"  link2="wrist2"  reason="Adjacent" />
11 <dis_col link1="wrist1"  link2="wrist2"  reason="never" />
12 <dis_col link1="wrist2"  link2="wrist3"  reason="Adjacent" />

```

Listing G.3: Myworkcell file which contains information about various planning groups joints and their types

```

1  <robot name="robot_example">
2    <link name="parent_link">
3      <visual>
4        <geometry>
5          <cylinder length="0.6" radius="0.2" />
6        </geometry>
7        <material name="gold">
8          <color rgba= (1,0.549,0,1) />
9        </material>
10     </visual>
11   </link>
12   <link name="child_link">
13     ...
14   </link>
15   <joint name="parent_to_child_link" type="fixed">
16     <parent link="parent_link" />
17     <child link="child_leg" />
18   </joint>
19 </robot>

```

Listing G.4: A snippet describing how to assign visual and geometry properties to a link and also how define a connection between two links via a joint.

```

1 1 state planning_time: "0.36788", execution_time: "8.92433"
2 2 state planning_time: "1.57946", execution_time: "10.1688"
3 3 state planning_time: "0.92870", execution_time: "11.3787"
4 4 state planning_time: "0.49580", execution_time: "8.77156"
5 5 state planning_time: "0.15313", execution_time: "6.31147"
6 6 state planning_time: "0.62722", execution_time: "9.30022"
7 7 state planning_time: "0.82149", execution_time: "9.76146"
8 8 state planning_time: "0.88676", execution_time: "8.07944"
9 9 state planning_time: "0.74044", execution_time: "7.94493"
10 10 state planning_time: "1.14723", execution_time: "8.36082"
11 ...
12 Total planning time: "28.7553"
13 Average planning time: "0.871372"
14 Total execution time: "305.231"
15 Average execution time: "9.24944"

```

Listing G.5: Planning time

```

1 Planner ID: RRTConnectkConfigDefault
2 1 state: (-8.1823e-5, -4.0411e-5, 3.9703e-5)
3 --> Total_diff = "9.95218e-05"
4 2 state: (-3.6621e-5, 6.6976e-5, 5.2046e-5)
5 --> Total_diff = "9.23892e-05"
6 3 state: (-3.0193e-5, 4.9272e-5, -5.7582e-5)
7 --> Total_diff = "8.15794e-05"
8 4 state: (-2.2148e-5, -4.7148e-6, 5.3853e-6)
9 --> Total_diff = "2.32761e-05"
10 5 state: (-1.2601e-5, 1.8013e-5, 2.6118e-5)
11 --> Total_diff = "3.41389e-05"
12 6 state: (-3.7357e-5, 1.6381e-5, -4.1905e-5)
13 --> Total_diff = "5.84807e-05"
14 7 state: (-3.8371e-5, 3.8582e-5, -7.0802e-5)
15 --> Total_diff = "8.92975e-05"
16 8 state: (-1.9581e-5, -3.4249e-5, -8.5127e-5)
17 --> Total_diff = "9.38255e-05"
18 9 state: (-5.3662e-5, 3.8743e-5, 1.6997e-5)
19 --> Total_diff = "6.83349e-05"
20 10 state: (6.7817e-6, 8.5335e-5, 2.0847e-5)
21 --> Total_diff = "8.81066e-05"
22 ...
23 Average diff = "7.79454e-05"
24 ***** Test finish *****

```

Listing G.6: Metric accuracy

```

1  geometry::_msgs::Pose diff_pose;
2  diff_pose.orientation.w = target_pose.orientation.w;
3  diff_pose.position.x = target_pose.position.x -
4  current_pose.position.x;
5  diff_pose.position.y = target_pose.position.y -
6  current_pose.position.y;
7  diff_pose.position.z = target_pose.position.z -
8  current_pose.position.z;
9  total_diff = sqrt(
10 diff_pose.position.x*diff_pose.position.x +
11 diff_pose.position.y*diff_pose.position.y +
12 diff_pose.position.z*diff_pose.position.z
13 );

```

Listing G.7: Calculation of planning accuracy.

```

1  [problem]
2  "name" = my_benchmark_problem
3  "robot" = my_robot_mesh.dae
4  "start.x, start.y, start.theta" = (0,0,0)
5  "goal.x, goal.y, goal.theta" = (1,-1, 0)
6  [benchmark]
7  "time_limit" = 10.0
8  "Memory limit" = 800
9  [planner]
10 "rtt" =

```

Listing G.8: A .yaml configuration file that contains information about the scenes and queries

```

1  #include <tf2/LinearMath/Quaternion.h>
2  tf2::Quaternion myQuaternion;
3  myQuaternion.setRPY(0,0,0);

```

Listing G.9: tf2::Quaternion object which can be used to transform Euler representation to quaternion representation.

TRITA -EECS-EX-2022:13